

CIS 6270

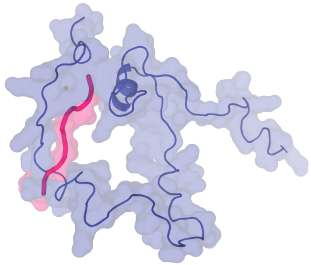
Module 4: Discrete Diffusion

Lecture 4.4: Discrete Diffusion Models and Guided Sequence Generation

Thu, Oct 15, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Summary so far:

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

The transition matrix $P_{t,t+h}$ over a short interval h equals the identity I plus h times the rate matrix R_t, up to o(h).

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The time derivative $\dot{p}_t$ of the probability vector equals p_t times the rate matrix R_t.

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

The transition matrix $P_{0,t}$ equals the matrix exponential of the clock τ(t) times the rate matrix R.

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

The stationary flux π(x)R(x, y) from x to y equals the flux π(y)R(y, x) from y back to x.

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The reverse rate $\hat{R}_t(y, x)$ equals the forward rate $R_t(x, y)$ times $p_t(x)$ divided by $p_t(y)$.

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The posterior averages conditional rates: $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The posterior averages conditional rates: $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$

The marginal reverse rate $\hat{R}_t(y, x)$ is the expectation of the conditional rate $\hat{R}_t(y, x \mid X_0)$ over clean sequences X_0, given that $X_t = y$.

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The posterior averages conditional rates: $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$

To condition, we use Doob's h-transform: $R_t^h(x,y) = R_t(x,y)\, h_t(y)/h_t(x)$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The posterior averages conditional rates: $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$

To condition, we use Doob's h-transform: $R_t^h(x,y) = R_t(x,y)\, h_t(y)/h_t(x)$

The conditioned rate $R^h_t(x, y)$ equals $R_t(x, y)$ times the value $h_t(y)$ divided by $h_t(x)$.

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The posterior averages conditional rates: $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$

To condition, we use Doob's h-transform: $R_t^h(x,y) = R_t(x,y)\, h_t(y)/h_t(x)$

With a rate loss, we learn the rates: $\int_0^1 \mathbb{E} \sum_y \ell\big(\hat{R}_t(\cdot \mid x_0), \hat{R}_t^\theta\big)\, dt$

Summary so far:

Rates describe short intervals: $P_{t,t+h} = I + hR_t + o(h)$

Probability follows the master equation: $\dot{p}_t = p_t R_t$

The matrix exponential solves it: $P_{0,t} = e^{\tau(t)R}$

Detailed balance holds on each edge: $\pi(x)R(x,y) = \pi(y)R(y,x)$

To reverse, use probability ratios: $\hat{R}_t(y,x) = R_t(x,y)\, p_t(x)/p_t(y)$

The posterior averages conditional rates: $\hat{R}_t(y,x) = \mathbb{E}\big[\hat{R}_t(y,x \mid X_0) \mid X_t = y\big]$

To condition, we use Doob's h-transform: $R_t^h(x,y) = R_t(x,y)\, h_t(y)/h_t(x)$

With a rate loss, we learn the rates: $\int_0^1 \mathbb{E} \sum_y \ell\big(\hat{R}_t(\cdot \mid x_0), \hat{R}_t^\theta\big)\, dt$

The loss integrates over time the expected sum over destinations y of the rate error ℓ between the conditional reverse rate $\hat{R}_t(\cdot \mid x_0)$ and the model rate $\hat{R}^\theta_t$.

To build a discrete diffusion model, we need five facts

To build a discrete diffusion model, we need five facts

1. The forward rates are nonnegative off the diagonal, with zero row sums.

To build a discrete diffusion model, we need five facts

1. The forward rates are nonnegative off the diagonal, with zero row sums.
2. The forward marginal $q_t(z \mid x_0)$ has a closed form, so in training we can sample any noise level directly.

To build a discrete diffusion model, we need five facts

1. The forward rates are nonnegative off the diagonal, with zero row sums.
2. The forward marginal $q_t(z \mid x_0)$ has a closed form, so in training we can sample any noise level directly.
3. The forward process ends at a distribution we can sample, ideally a stationary one.

To build a discrete diffusion model, we need five facts

1. The forward rates are nonnegative off the diagonal, with zero row sums.
2. The forward marginal $q_t(z \mid x_0)$ has a closed form, so in training we can sample any noise level directly.
3. The forward process ends at a distribution we can sample, ideally a stationary one.
4. The reverse rates follow from the posterior over clean tokens, one position at a time.

To build a discrete diffusion model, we need five facts

1. The forward rates are nonnegative off the diagonal, with zero row sums.
2. The forward marginal $q_t(z \mid x_0)$ has a closed form, so in training we can sample any noise level directly.
3. The forward process ends at a distribution we can sample, ideally a stationary one.
4. The reverse rates follow from the posterior over clean tokens, one position at a time.
5. The loss bounds the likelihood and is minimized at the true reverse rates.

To build a discrete diffusion model, we need five facts

1. The forward rates are nonnegative off the diagonal, with zero row sums.
2. The forward marginal $q_t(z \mid x_0)$ has a closed form, so in training we can sample any noise level directly.
3. The forward process ends at a distribution we can sample, ideally a stationary one.
4. The reverse rates follow from the posterior over clean tokens, one position at a time.
5. The loss bounds the likelihood and is minimized at the true reverse rates.

Every model in the rest of this lecture is one specific answer to these five questions.

Now take six models, one at a time

Now take six models, one at a time

Each model below is one choice of rate matrix, and from the framework we get its reverse rates and its loss.

Now take six models, one at a time

Each model below is one choice of rate matrix, and from the framework we get its reverse rates and its loss.

For each one, we will introduce the paper, check the five facts, and then examine the results.

Let’s start with the model we derived by hand

Each model below is one choice of rate matrix, and from the framework we get its reverse rates and its loss.

For each one, we will introduce the paper, check the five facts, and then examine the results.

What do our five conditions give us when every token can only become a mask?

MDLM

Simple and Effective Masked Diffusion Language Models

Subham Sekhar Sahoo
Cornell Tech, NYC, USA.
ssahoo@cs.cornell.edu

Marianne Arriola
Cornell Tech, NYC, USA.
ma2238@cornell.edu

Yair Schiff
Cornell Tech, NYC, USA.
yzs2@cornell.edu

Aaron Gokaslan
Cornell Tech, NYC, USA.
akg87@cs.cornell.edu

Edgar Marroquin
Cornell Tech, NYC, USA.
emm392@cornell.edu

Justin T Chiu
Cornell Tech, NYC, USA.
jtc257@cornell.edu

Alexander Rush
Cornell Tech, NYC, USA.
ar459@cornell.edu

Volodymyr Kuleshov
Cornell Tech, NYC, USA.
kuleshov@cornell.edu

Sahoo,..,Kuleshov. *NeurIPS* (2024)

For MDLM, we choose $\pi = \delta_m$

For MDLM, we choose $\pi = \delta_m$

Each clean token jumps to the mask at rate β(t), and the mask never leaves:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

For MDLM, we choose $\pi = \delta_m$

Each clean token jumps to the mask at rate β(t), and the mask never leaves:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

The rate $R_t(a, m)$ from every clean token a to the mask token m equals β(t), and the rate $R_t(m, a)$ from the mask back to a is zero.

For MDLM, we choose $\pi = \delta_m$

Each clean token jumps to the mask at rate β(t), and the mask never leaves:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

The rate $R_t(a, m)$ from every clean token a to the mask token m equals β(t), and the rate $R_t(m, a)$ from the mask back to a is zero.

Fact 1: the rates are nonnegative off the diagonal, and each row sums to zero.

For MDLM, we choose $\pi = \delta_m$

Each clean token jumps to the mask at rate β(t), and the mask never leaves:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

The rate $R_t(a, m)$ from every clean token a to the mask token m equals β(t), and the rate $R_t(m, a)$ from the mask back to a is zero.

Fact 1: the rates are nonnegative off the diagonal, and each row sums to zero.

Fact 2: the matrix exponential is the masking marginal from the start of the lecture:

$$q_t(z^i \mid x_0^i) = \alpha_t \, \delta(z^i, x_0^i) + (1 - \alpha_t) \, \delta(z^i, m)$$

For MDLM, we choose $\pi = \delta_m$

Each clean token jumps to the mask at rate β(t), and the mask never leaves:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

The rate $R_t(a, m)$ from every clean token a to the mask token m equals β(t), and the rate $R_t(m, a)$ from the mask back to a is zero.

Fact 1: the rates are nonnegative off the diagonal, and each row sums to zero.

Fact 2: the matrix exponential is the masking marginal from the start of the lecture:

$$q_t(z^i \mid x_0^i) = \alpha_t \, \delta(z^i, x_0^i) + (1 - \alpha_t) \, \delta(z^i, m)$$

The probability $q_t(z^i \mid x_0{}^i)$ of the noisy token z^i given the clean token $x_0{}^i$ equals the survival probability α_t times $\delta(z^i, x_0{}^i)$ plus one minus α_t times $\delta(z^i, m)$, the indicator that position i is masked.

For MDLM, we choose $\pi = \delta_m$

Each clean token jumps to the mask at rate β(t), and the mask never leaves:

$$R_t(a, m) = \beta(t), \qquad R_t(m, a) = 0 \quad \text{for every clean token } a$$

The rate $R_t(a, m)$ from every clean token a to the mask token m equals β(t), and the rate $R_t(m, a)$ from the mask back to a is zero.

Fact 1: the rates are nonnegative off the diagonal, and each row sums to zero.

Fact 2: the matrix exponential is the masking marginal from the start of the lecture:

$$q_t(z^i \mid x_0^i) = \alpha_t \, \delta(z^i, x_0^i) + (1 - \alpha_t) \, \delta(z^i, m)$$

The probability $q_t(z^i \mid x_0^i)$ of the noisy token z^i given the clean token x_0^i equals the survival probability α_t times $\delta(z^i, x_0^i)$ plus one minus α_t times $\delta(z^i, m)$, the indicator that position i is masked.

Fact 3: as $\alpha_t \to 0$ every position is masked, so generation starts from the all-mask sequence.

Now compute the reverse rates for masking

Now compute the reverse rates for masking

At a masked position, the conditional reverse rate toward clean token a is

$$\hat{R}_t(m \to a \mid x_0) = R_t(a, m)\,\frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t)\,\frac{\alpha_t\,\delta(a, x_0)}{1 - \alpha_t}$$

Now compute the reverse rates for masking

At a masked position, the conditional reverse rate toward clean token a is

$$\hat{R}_t(m \to a \mid x_0) = R_t(a, m)\,\frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t)\,\frac{\alpha_t\,\delta(a, x_0)}{1 - \alpha_t}$$

The conditional reverse rate $\hat{R}_t(m \to a \mid x_0)$ from the mask m to token a equals the forward rate $R_t(a, m)$ times $q_t(a \mid x_0)$ divided by $q_t(m \mid x_0)$, which is $\beta(t)$ times $\alpha_t \delta(a, x_0)$ divided by one minus α_t.

Now compute the reverse rates for masking

At a masked position, the conditional reverse rate toward clean token a is

$$\hat{R}_t(m \to a \mid x_0) = R_t(a, m)\,\frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t)\,\frac{\alpha_t\,\delta(a, x_0)}{1 - \alpha_t}$$

Substitute $\beta(t) = -\dot{\alpha}_t/\alpha_t$, then average with the posterior, using the denoiser:

$$\boxed{\hat{R}^{\theta}_t(m \to a) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}\,d^{i}_{\theta}(a \mid z, t)}$$

Now compute the reverse rates for masking

At a masked position, the conditional reverse rate toward clean token a is

$$\hat{R}_t(m \to a \mid x_0) = R_t(a, m)\,\frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t)\,\frac{\alpha_t\,\delta(a, x_0)}{1 - \alpha_t}$$

Substitute $\beta(t) = -\dot{\alpha}_t/\alpha_t$, then average with the posterior, using the denoiser:

$$\boxed{\hat{R}^{\theta}_t(m \to a) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}\,d^i_\theta(a \mid z, t)}$$

The model rate $\hat{R}^{\theta}_t(m \to a)$ of revealing token a at a masked position i equals negative $\dot{\alpha}_t$ divided by one minus α_t, times the denoiser probability $d^i_\theta(a \mid z, t)$ of token a at position i given the noisy sequence z and the time t.

Now compute the reverse rates for masking

At a masked position, the conditional reverse rate toward clean token a is

$$\hat{R}_t(m \to a \mid x_0) = R_t(a, m)\,\frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t)\,\frac{\alpha_t\,\delta(a, x_0)}{1 - \alpha_t}$$

Substitute $\beta(t) = -\dot{\alpha}_t/\alpha_t$, then average with the posterior, using the denoiser:

$$\boxed{\hat{R}^{\theta}_t(m \to a) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}\,d^i_\theta(a \mid z, t)}$$

The model rate $\hat{R}^{\theta}_t(m \to a)$ of revealing token a at a masked position i equals negative $\dot{\alpha}_t$ divided by one minus α_t, times the denoiser probability $d^i_\theta(a \mid z, t)$ of token a at position i given the noisy sequence z and the time t.

The reveal rate $-\dot{\alpha}_t/(1 - \alpha_t)$ is the loss weight w(t) we found from the discrete-time bound.

Now compute the reverse rates for masking

At a masked position, the conditional reverse rate toward clean token a is

$$\hat{R}_t(m \to a \mid x_0) = R_t(a, m)\,\frac{q_t(a \mid x_0)}{q_t(m \mid x_0)} = \beta(t)\,\frac{\alpha_t\,\delta(a, x_0)}{1 - \alpha_t}$$

Substitute $\beta(t) = -\dot{\alpha}_t/\alpha_t$, then average with the posterior, using the denoiser:

$$\boxed{\hat{R}^{\theta}_t(m \to a) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}\,d^i_\theta(a \mid z, t)}$$

The model rate $\hat{R}^{\theta}_t(m \to a)$ of revealing token a at a masked position i equals negative $\dot{\alpha}_t$ divided by one minus α_t, times the denoiser probability $d^i_\theta(a \mid z, t)$ of token a at position i given the noisy sequence z and the time t.

The schedule decides how quickly masks disappear; the denoiser decides which token each mask becomes.

Suppose a position is already visible

Suppose a position is already visible

At a visible position holding token a, a reverse jump to y is possible only if there is a forward jump from y into a.

Suppose a position is already visible

At a visible position holding token a, a reverse jump to y is possible only if there is a forward jump from y into a.

Masking never jumps into a clean token, so every forward rate into a is zero, and so is every reverse rate out of a.

Suppose a position is already visible

At a visible position holding token a, a reverse jump to y is possible only if there is a forward jump from y into a.

Masking never jumps into a clean token, so every forward rate into a is zero, and so is every reverse rate out of a.

That is MDLM’s carry-over unmasking, now derived from the rates.

Suppose a position is already visible

At a visible position holding token a, a reverse jump to y is possible only if there is a forward jump from y into a.

Masking never jumps into a clean token, so every forward rate into a is zero, and so is every reverse rate out of a.

That is MDLM’s carry-over unmasking, now derived from the rates.

The mask is never a clean token, so the posterior has no probability on m, which is the other SUBS rule.

Let’s look at AmGm halfway through generation

$$\hat{R}_t^\theta(m \to a) = w(t)\, d_\theta^i(a \mid z, t)$$

$$w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}$$

numbers: denoiser probabilities



The two masked positions are the only ones that can change

$$\hat{R}_t^\theta(m \to a) = w(t)\, d_\theta^i(a \mid z, t)$$

$$w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}$$

numbers: denoiser probabilities

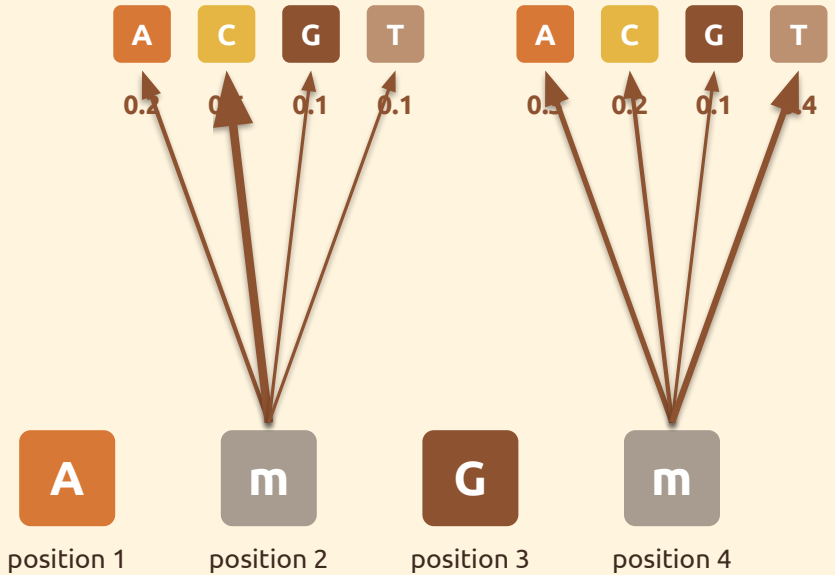


Each reveals at total rate w(t), split over the four bases in proportion to the denoiser probabilities

$$\hat{R}_t^\theta(m \to a) = w(t)\, d_\theta^i(a \mid z, t)$$

$$w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}$$

numbers: denoiser probabilities

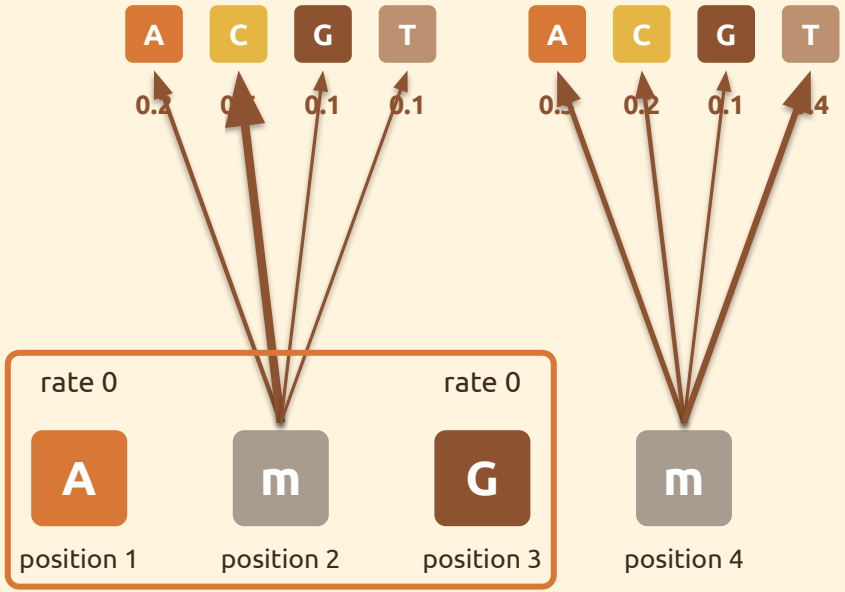


The visible A and G have rate zero in every direction

$$\hat{R}_t^\theta(m \to a) = w(t)\, d_\theta^i(a \mid z, t)$$

$$w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}$$

numbers: denoiser probabilities

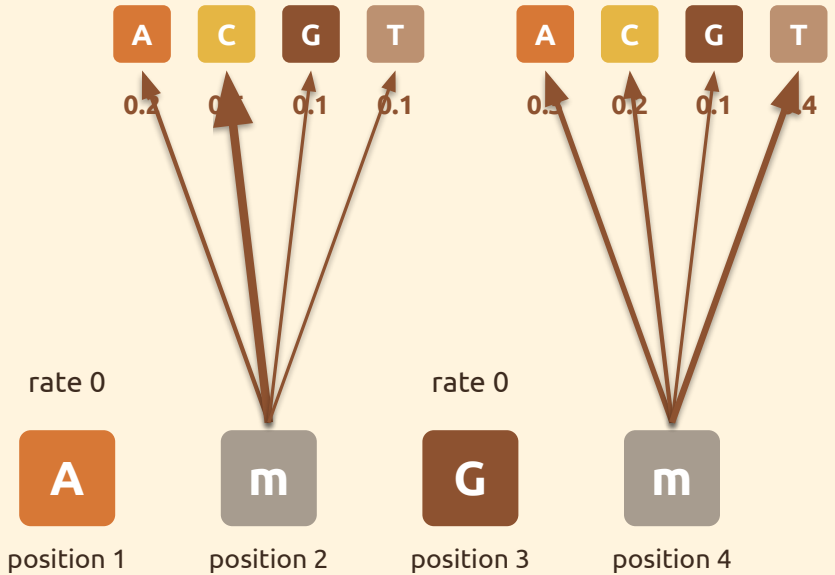


A masked position can only reveal, and a visible position never changes

$$\hat{R}_t^\theta(m \to a) = w(t)\, d_\theta^i(a \mid z, t)$$

$$w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t}$$

numbers: denoiser probabilities



Finally, we compute the rate loss for MDLM

Finally, we compute the rate loss for MDLM

At a masked position, the target rates are $w(t)\delta(a, x_0^i)$ and the model rates are $w(t)d_\theta^i(a \mid z, t)$.

Finally, we compute the rate loss for MDLM

At a masked position, the target rates are $w(t)\delta(a, x_0^i)$ and the model rates are $w(t)d_\theta^i(a \mid z, t)$.

Substitute into the rate error and sum over a:

$$\sum_a \ell = w(t)\Big[-\log d_\theta^i(x_0^i \mid z, t) - 1 + \sum_a d_\theta^i(a \mid z, t)\Big] = w(t)\big[-\log d_\theta^i(x_0^i \mid z, t)\big]$$

Finally, we compute the rate loss for MDLM

At a masked position, the target rates are $w(t)\delta(a, x_0^i)$ and the model rates are $w(t)d_\theta^i(a \mid z, t)$.

Substitute into the rate error and sum over a:

$$\sum_a \ell = w(t)\Big[-\log d_\theta^i(x_0^i \mid z, t) - 1 + \sum_a d_\theta^i(a \mid z, t)\Big] = w(t)\big[-\log d_\theta^i(x_0^i \mid z, t)\big]$$

The sum over tokens a of the rate error ℓ at a masked position equals w(t) times the quantity negative log $d^i_\theta(x^i_0 \mid z, t)$ minus one plus the sum of the denoiser probabilities $d^i_\theta(a \mid z, t)$, and those probabilities sum to one, so it equals w(t) times negative log $d^i_\theta(x^i_0 \mid z, t)$.

Finally, we compute the rate loss for MDLM

At a masked position, the target rates are $w(t)\delta(a, x_0^i)$ and the model rates are $w(t)d_\theta^i(a \mid z, t)$.

Substitute into the rate error and sum over a:

$$\sum_a \ell = w(t)\Big[-\log d_\theta^i(x_0^i \mid z, t) - 1 + \sum_a d_\theta^i(a \mid z, t)\Big] = w(t)\big[-\log d_\theta^i(x_0^i \mid z, t)\big]$$

The sum over tokens a of the rate error ℓ at a masked position equals w(t) times the quantity negative log $d^i_\theta(x^i_0 \mid z, t)$ minus one plus the sum of the denoiser probabilities $d^i_\theta(a \mid z, t)$, and those probabilities sum to one, so it equals w(t) times negative log $d^i_\theta(x^i_0 \mid z, t)$.

At a visible position, every target rate and every model rate is zero, so the error is zero.

That is the MDLM loss

That is the MDLM loss

Add the masked positions and integrate over time:

$$\mathcal{L}_{\mathrm{MDLM}} = \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}_{x_0,\, Z_t}\Big[\sum_{i:\, Z_t^i = m} -\log d_\theta^i\big(x_0^i \mid Z_t, t\big)\Big]\, dt$$

That is the MDLM loss

Add the masked positions and integrate over time:

$$\mathcal{L}_{\mathrm{MDLM}} = \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}_{x_0,\, Z_t}\Big[\sum_{i:\, Z_t^i = m} -\log d_\theta^i\big(x_0^i \mid Z_t, t\big)\Big]\, dt$$

The MDLM loss L_{MDLM} equals the integral over time t from 0 to 1 of the weight negative $\dot{\alpha}_t$ divided by one minus α_t, times the expectation over clean sequences x_0 and noisy sequences Z_t of the sum over masked positions i of negative log $d^i_\theta(x^i_0 \mid Z_t, t)$, the denoiser's negative log probability of the clean token x^i_0.

That is the MDLM loss

Add the masked positions and integrate over time:

$$\mathcal{L}_{\text{MDLM}} = \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}_{x_0,\, Z_t}\Big[\sum_{i:\, Z_t^i = m} -\log d_\theta^i\big(x_0^i \mid Z_t, t\big)\Big]\, dt$$

The MDLM loss L_{MDLM} equals the integral over time t from 0 to 1 of the weight negative $\dot{\alpha}_t$ divided by one minus α_t, times the expectation over clean sequences x_0 and noisy sequences Z_t of the sum over masked positions i of negative log $d^i_\theta(x^i_0 \mid Z_t, t)$, the denoiser's negative log probability of the clean token x^i_0.

This is the weighted cross-entropy we derived one reverse step at a time, and from the rate loss we get it in one line.

That is the MDLM loss

Add the masked positions and integrate over time:

$$\mathcal{L}_{\mathrm{MDLM}} = \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}_{x_0,\, Z_t}\Big[\sum_{i:\, Z_t^i = m} -\log d_\theta^i\big(x_0^i \mid Z_t, t\big)\Big]\, dt$$

The MDLM loss L_{MDLM} equals the integral over time t from 0 to 1 of the weight negative $\dot{\alpha}_t$ divided by one minus α_t, times the expectation over clean sequences x_0 and noisy sequences Z_t of the sum over masked positions i of negative log $d^i_\theta(x^i_0 \mid Z_t, t)$, the denoiser's negative log probability of the clean token x^i_0.

This is the weighted cross-entropy we derived one reverse step at a time, and from the rate loss we get it in one line.

Facts 4 and 5 hold, so MDLM is the $\pi = \delta_m$ member of our family.

That is the MDLM loss

Add the masked positions and integrate over time:

$$\mathcal{L}_{\text{MDLM}} = \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}_{x_0,\, Z_t}\Big[\sum_{i:\, Z_t^i = m} -\log d_\theta^i\big(x_0^i \mid Z_t, t\big)\Big]\, dt$$

The MDLM loss L_{MDLM} equals the integral over time t from 0 to 1 of the weight negative $\dot{\alpha}_t$ divided by one minus α_t, times the expectation over clean sequences x_0 and noisy sequences Z_t of the sum over masked positions i of negative log $d^i_\theta(x^i_0 \mid Z_t, t)$, the denoiser's negative log probability of the clean token x^i_0.

This is the weighted cross-entropy we derived one reverse step at a time, and from the rate loss we get it in one line.

Every hidden token becomes a prediction problem, weighted by how quickly that mask should be revealed.

MDLM on A [MASK] G T

Toy DNA target ACGT. The denoiser gives the missing C probability 0.6. Use the linear survival schedule, with current time 0.75 and earlier time 0.5.

MDLM on A [MASK] G T

Toy DNA target ACGT. The denoiser gives the missing C probability 0.6. Use the linear survival schedule, with current time 0.75 and earlier time 0.5.

$$p_\theta(Z^2_{0.5} = \mathrm{C} \mid Z_{0.75}) = \frac{0.5 - 0.25}{1 - 0.25}\,(0.6) = 0.2$$

MDLM on A [MASK] G T

Toy DNA target ACGT. The denoiser gives the missing C probability 0.6. Use the linear survival schedule, with current time 0.75 and earlier time 0.5.

$$p_\theta(Z^2_{0.5} = \mathrm{C} \mid Z_{0.75}) = \frac{0.5 - 0.25}{1 - 0.25}\,(0.6) = 0.2$$

One third of the masks open in this reverse step. C receives 60% of that probability.

MDLM on A [MASK] G T

Toy DNA target ACGT. The denoiser gives the missing C probability 0.6. Use the linear survival schedule, with current time 0.75 and earlier time 0.5.

$$p_\theta(Z_{0.5}^2 = \mathrm{C} \mid Z_{0.75}) = \frac{0.5 - 0.25}{1 - 0.25}(0.6) = 0.2$$

One third of the masks open in this reverse step. C receives 60% of that probability.

$$\ell_{0.75} = \frac{-\log(0.6)}{0.75} \approx 0.681$$

MDLM on A [MASK] G T

Toy DNA target ACGT. The denoiser gives the missing C probability 0.6. Use the linear survival schedule, with current time 0.75 and earlier time 0.5.

$$p_\theta(Z_{0.5}^2 = \mathrm{C} \mid Z_{0.75}) = \frac{0.5 - 0.25}{1 - 0.25}\,(0.6) = 0.2$$

One third of the masks open in this reverse step. C receives 60% of that probability.

$$\ell_{0.75} = \frac{-\log(0.6)}{0.75} \approx 0.681$$

At this time, the masked C contributes 0.681 to the weighted cross-entropy integrand.

MDLM on A [MASK] G T

Toy DNA target ACGT. The denoiser gives the missing C probability 0.6. Use the linear survival schedule, with current time 0.75 and earlier time 0.5.

$$p_\theta(Z_{0.5}^2 = \mathrm{C} \mid Z_{0.75}) = \frac{0.5 - 0.25}{1 - 0.25}(0.6) = 0.2$$

One third of the masks open in this reverse step. C receives 60% of that probability.

$$\ell_{0.75} = \frac{-\log(0.6)}{0.75} \approx 0.681$$

At this time, the masked C contributes 0.681 to the weighted cross-entropy integrand.

The schedule decides when to reveal. The denoiser decides which base appears.

In MDLM, we use masking, a denoiser, and the weighted cross-entropy we just derived

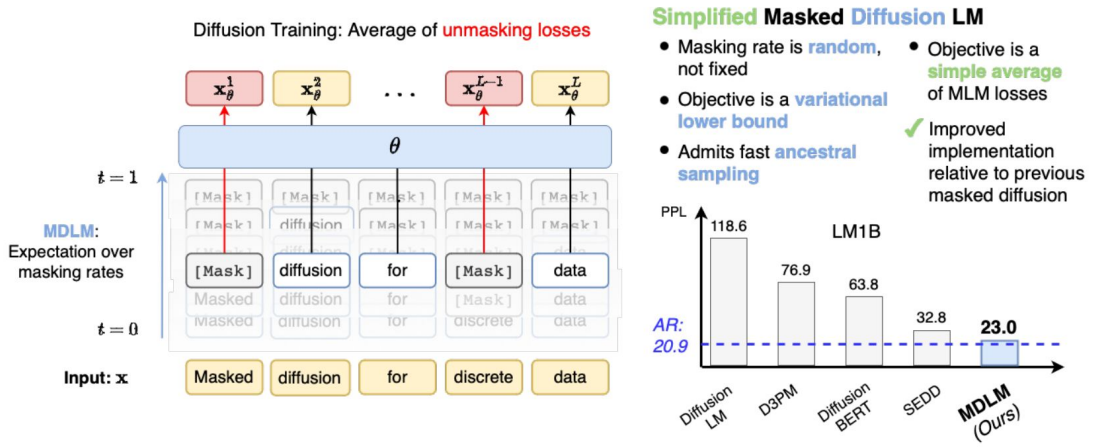


Figure 1: *(Left)* Our proposed masked diffusion language model (MDLM) is trained using a weighted average of masked cross entropy losses. (*Top Right*) In comparison to masked language models (MLM), MDLM's objective correspond to a principled variational lower bound, and supports generation via ancestral sampling. (*Bottom Right*) Perplexity (PPL) on One Billion Words (LM1B) benchmark.

The SUBS parameterization is the carry-over and zero-mask rule we read off the rates.

Sahoo,..,Kuleshov. *NeurIPS* (2024)

Here is the LM1B result in Table 1 of the paper

Table 1: Test perplexities (PPL; ↓) on LM1B. †Reported in He et al. [26]. Best diffusion value is bolded.

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

Sahoo,..,Kuleshov. *NeurIPS* (2024)

A retrained transformer reaches 20.86 on LM1B

Table 1: Test perplexities (PPL; ↓) on LM1B. †Reported in He et al. [26]. Best diffusion value is bolded.

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

Sahoo,..,Kuleshov. *NeurIPS* (2024)

MDLM has a perplexity bound of 23.00 with the same 327B tokens

Table 1: Test perplexities (PPL; ↓) on LM1B. †Reported in He et al. [26]. Best diffusion value is bolded.

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

Sahoo,..,Kuleshov. *NeurIPS* (2024)

SEDD, the ratio-based model we meet shortly, reaches 32.79

Table 1: Test perplexities (PPL; ↓) on LM1B. †Reported in He et al. [26]. Best diffusion value is bolded.

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

Sahoo,..,Kuleshov. *NeurIPS* (2024)

Masked diffusion came within three points of autoregression on LM1B

Table 1: Test perplexities (PPL; ↓) on LM1B. †Reported in He et al. [26]. Best diffusion value is bolded.

		Parameters	PPL (↓)
Autoregressive	Transformer-X Base [13]	0.46B	23.5
	OmniNet$_T$ [61]	100M	21.5
Diffusion	BERT-Mouth [64]†	110M	≤142.89
	D3PM (absorb) [1]	70M	≤76.90
	Diffusion-LM [30]†	80M	≤118.62
	DiffusionBert [26]	110M	≤63.78
	SEDD [33] (33B tokens)	110M	≤ 32.79
Autoregressive (Retrained)	Transformer (33B tokens)	110M	22.32
	Transformer (327B tokens)		20.86
Diffusion (Ours)	MDLM (33B tokens)	110M	≤27.04
	MDLM (327B tokens)		≤**23.00**

Sahoo,..,Kuleshov. *NeurIPS* (2024)

Could we take finite corruption steps and choose a transition matrix for each step?

D3PM

Structured Denoising Diffusion Models in Discrete State-Spaces

Jacob Austin,* Daniel D. Johnson,* Jonathan Ho, Daniel Tarlow & Rianne van den Berg†
Google Research, Brain Team
{jaaustin,ddjohnson,jonathanho,dtarlow,riannevdberg}@google.com

Austin,..,van den Berg. *NeurIPS* (2021)

Now let time be discrete, as in D3PM

Now let time be discrete, as in D3PM

D3PM was the first general discrete diffusion model, two years before the continuous-time limit was taken.

Now let time be discrete, as in D3PM

D3PM was the first general discrete diffusion model, two years before the continuous-time limit was taken.

In D3PM, we corrupt with one transition matrix per step, and multiply them:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ x_{t-1}Q_t\big), \qquad q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ x_0\bar{Q}_t\big)$$

Now let time be discrete, as in D3PM

D3PM was the first general discrete diffusion model, two years before the continuous-time limit was taken.

In D3PM, we corrupt with one transition matrix per step, and multiply them:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ x_{t-1}Q_t\big), \qquad q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ x_0\bar{Q}_t\big)$$

The one-step distribution $q(x_t \mid x_{t-1})$ is categorical with probabilities $x_{t-1}Q_t$, the one-hot row x_{t-1} times the transition matrix Q_t, and $q(x_t \mid x_0)$ is categorical with probabilities $x_0\bar{Q}_t$, where $\bar{Q}_t$ is the product $Q_1Q_2\cdots Q_t$.

Now let time be discrete, as in D3PM

D3PM was the first general discrete diffusion model, two years before the continuous-time limit was taken.

In D3PM, we corrupt with one transition matrix per step, and multiply them:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ x_{t-1}Q_t\big), \qquad q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ x_0\bar{Q}_t\big)$$

The one-step distribution $q(x_t \mid x_{t-1})$ is categorical with probabilities $x_{t-1}Q_t$, the one-hot row x_{t-1} times the transition matrix Q_t, and $q(x_t \mid x_0)$ is categorical with probabilities $x_0\bar{Q}_t$, where $\bar{Q}_t$ is the product $Q_1Q_2\cdots Q_t$.

Choose $Q_t = (1 - \beta_t)I + \beta_t\mathbf{1}\pi^\top$. Because $M^2 = M$, products stay in the family:

$$\big(aI + (1-a)M\big)\big(cI + (1-c)M\big) = ac\,I + (1-ac)\,M$$

Now let time be discrete, as in D3PM

D3PM was the first general discrete diffusion model, two years before the continuous-time limit was taken.

In D3PM, we corrupt with one transition matrix per step, and multiply them:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ x_{t-1}Q_t\big), \qquad q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ x_0\bar{Q}_t\big)$$

The one-step distribution $q(x_t \mid x_{t-1})$ is categorical with probabilities $x_{t-1}Q_t$, the one-hot row x_{t-1} times the transition matrix Q_t, and $q(x_t \mid x_0)$ is categorical with probabilities $x_0\bar{Q}_t$, where $\bar{Q}_t$ is the product $Q_1Q_2\cdots Q_t$.

Choose $Q_t = (1 - \beta_t)I + \beta_t\mathbf{1}\pi^\top$. Because $M^2 = M$, products stay in the family:

$$\big(aI + (1-a)M\big)\big(cI + (1-c)M\big) = ac\,I + (1-ac)\,M$$

The product of aI + (1 – a)M and cI + (1 – c)M equals ac times I plus 1 – ac times M.

Now let time be discrete, as in D3PM

D3PM was the first general discrete diffusion model, two years before the continuous-time limit was taken.

In D3PM, we corrupt with one transition matrix per step, and multiply them:

$$q(x_t \mid x_{t-1}) = \mathrm{Cat}\big(x_t;\ x_{t-1}Q_t\big), \qquad q(x_t \mid x_0) = \mathrm{Cat}\big(x_t;\ x_0\bar{Q}_t\big)$$

The one-step distribution $q(x_t \mid x_{t-1})$ is categorical with probabilities $x_{t-1}Q_t$, the one-hot row x_{t-1} times the transition matrix Q_t, and $q(x_t \mid x_0)$ is categorical with probabilities $x_0\bar{Q}_t$, where $\bar{Q}_t$ is the product $Q_1Q_2\cdots Q_t$.

Choose $Q_t = (1 - \beta_t)I + \beta_t\mathbf{1}\pi^\top$. Because $M^2 = M$, products stay in the family:

$$\big(aI + (1-a)M\big)\big(cI + (1-c)M\big) = ac\,I + (1-ac)\,M$$

When we multiply two of these matrices, their survival probabilities multiply.

Multiply these matrices over many steps

Multiply these matrices over many steps

After t steps,

$$\bar{Q}_t = \bar{\alpha}_t \, I + (1 - \bar{\alpha}_t) \, \mathbf{1}\pi^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

Multiply these matrices over many steps

After t steps,

$$\bar{Q}_t = \bar{\alpha}_t \, I + (1 - \bar{\alpha}_t) \, \mathbf{1}\pi^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

The cumulative matrix $\bar{Q}_t$ equals $\bar{\alpha}_t$ times the identity matrix I plus one minus $\bar{\alpha}_t$ times $\mathbf{1}\pi^\top$, the matrix whose rows all equal π, where $\bar{\alpha}_t$ is the product over steps s from 1 to t of the survival probabilities $1 - \beta_s$.

Multiply these matrices over many steps

After t steps,

$$\bar{Q}_t = \bar{\alpha}_t\, I + (1 - \bar{\alpha}_t)\, \mathbf{1}\pi^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

The cumulative matrix $\bar{Q}_t$ equals $\bar{\alpha}_t$ times the identity matrix I plus one minus $\bar{\alpha}_t$ times $\mathbf{1}\pi^\top$, the matrix whose rows all equal π, where $\bar{\alpha}_t$ is the product over steps s from 1 to t of the survival probabilities $1 - \beta_s$.

With $\beta_s = \beta(s)\Delta t$, the product of survival probabilities becomes $e^{-\tau(t)}$, and $\bar{Q}_t$ becomes the matrix exponential of $\tau(t)R_\pi$.

Multiply these matrices over many steps

After t steps,

$$\bar{Q}_t = \bar{\alpha}_t\, I + (1 - \bar{\alpha}_t)\, \mathbf{1}\pi^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

The cumulative matrix $\bar{Q}_t$ equals $\bar{\alpha}_t$ times the identity matrix I plus one minus $\bar{\alpha}_t$ times $\mathbf{1}\pi^\top$, the matrix whose rows all equal π, where $\bar{\alpha}_t$ is the product over steps s from 1 to t of the survival probabilities $1 - \beta_s$.

With $\beta_s = \beta(s)\Delta t$, the product of survival probabilities becomes $e^{-\tau(t)}$, and $\bar{Q}_t$ becomes the matrix exponential of $\tau(t)R_\pi$.

Absorbing D3PM is masking, and uniform D3PM is uniform replacement.

Multiply these matrices over many steps

After t steps,

$$\bar{Q}_t = \bar{\alpha}_t\, I + (1 - \bar{\alpha}_t)\, \mathbf{1}\pi^\top, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1 - \beta_s)$$

The cumulative matrix $\bar{Q}_t$ equals $\bar{\alpha}_t$ times the identity matrix I plus one minus $\bar{\alpha}_t$ times $\mathbf{1}\pi^\top$, the matrix whose rows all equal π, where $\bar{\alpha}_t$ is the product over steps s from 1 to t of the survival probabilities $1 - \beta_s$.

With $\beta_s = \beta(s)\Delta t$, the product of survival probabilities becomes $e^{-\tau(t)}$, and $\bar{Q}_t$ becomes the matrix exponential of $\tau(t)R_\pi$.

Surviving all corruption steps requires surviving each one, so the survival probabilities multiply.

Next, we reverse one step with Bayes’ rule

Next, we reverse one step with Bayes' rule

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$$

Next, we reverse one step with Bayes' rule

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$$

The posterior $q(x_{t-1} \mid x_t, x_0)$ is categorical with probabilities $x_t Q_t^\top$, the probability of stepping from each earlier token to the observed token x_t, multiplied elementwise by $x_0 \bar{Q}_{t-1}$, the probability of reaching each earlier token from the clean token x_0, divided by $x_0 \bar{Q}_t x_t^\top$, the probability of x_t given x_0.

Next, we reverse one step with Bayes' rule

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$$

The posterior $q(x_{t-1} \mid x_t, x_0)$ is categorical with probabilities $x_t Q_t^\top$, the probability of stepping from each earlier token to the observed token x_t, multiplied elementwise by $x_0 \bar{Q}_{t-1}$, the probability of reaching each earlier token from the clean token x_0, divided by $x_0 \bar{Q}_t x_t^\top$, the probability of x_t given x_0.

The model averages that posterior over the denoiser's prediction:

$$p_\theta(x_{t-1} \mid x_t) \propto \sum_{\tilde{x}_0} q(x_{t-1}, x_t \mid \tilde{x}_0)\, \tilde{p}_\theta(\tilde{x}_0 \mid x_t)$$

Next, we reverse one step with Bayes' rule

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$$

The posterior $q(x_{t-1} \mid x_t, x_0)$ is categorical with probabilities $x_t Q_t^\top$, the probability of stepping from each earlier token to the observed token x_t, multiplied elementwise by $x_0 \bar{Q}_{t-1}$, the probability of reaching each earlier token from the clean token x_0, divided by $x_0 \bar{Q}_t x_t^\top$, the probability of x_t given x_0.

The model averages that posterior over the denoiser's prediction:

$$p_\theta(x_{t-1} \mid x_t) \propto \sum_{\tilde{x}_0} q(x_{t-1}, x_t \mid \tilde{x}_0)\, \tilde{p}_\theta(\tilde{x}_0 \mid x_t)$$

The model's reverse step $p_\theta(x_{t-1} \mid x_t)$ is proportional to the sum over predicted clean tokens $\tilde{x}_0$ of the joint probability $q(x_{t-1}, x_t \mid \tilde{x}_0)$ times the denoiser probability $\tilde{p}_\theta(\tilde{x}_0 \mid x_t)$.

Next, we reverse one step with Bayes' rule

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$$

The posterior $q(x_{t-1} \mid x_t, x_0)$ is categorical with probabilities $x_t Q_t^\top$, the probability of stepping from each earlier token to the observed token x_t, multiplied elementwise by $x_0 \bar{Q}_{t-1}$, the probability of reaching each earlier token from the clean token x_0, divided by $x_0 \bar{Q}_t x_t^\top$, the probability of x_t given x_0.

The model averages that posterior over the denoiser's prediction:

$$p_\theta(x_{t-1} \mid x_t) \propto \sum_{\tilde{x}_0} q(x_{t-1}, x_t \mid \tilde{x}_0)\, \tilde{p}_\theta(\tilde{x}_0 \mid x_t)$$

The model's reverse step $p_\theta(x_{t-1} \mid x_t)$ is proportional to the sum over predicted clean tokens $\tilde{x}_0$ of the joint probability $q(x_{t-1}, x_t \mid \tilde{x}_0)$ times the denoiser probability $\tilde{p}_\theta(\tilde{x}_0 \mid x_t)$.

This is our posterior average, written for a finite step.

Next, we reverse one step with Bayes' rule

$$q(x_{t-1} \mid x_t, x_0) = \mathrm{Cat}\left(x_{t-1};\ \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$$

The posterior $q(x_{t-1} \mid x_t, x_0)$ is categorical with probabilities $x_t Q_t^\top$, the probability of stepping from each earlier token to the observed token x_t, multiplied elementwise by $x_0 \bar{Q}_{t-1}$, the probability of reaching each earlier token from the clean token x_0, divided by $x_0 \bar{Q}_t x_t^\top$, the probability of x_t given x_0.

The model averages that posterior over the denoiser's prediction:

$$p_\theta(x_{t-1} \mid x_t) \propto \sum_{\tilde{x}_0} q(x_{t-1}, x_t \mid \tilde{x}_0)\, \tilde{p}_\theta(\tilde{x}_0 \mid x_t)$$

The model's reverse step $p_\theta(x_{t-1} \mid x_t)$ is proportional to the sum over predicted clean tokens $\tilde{x}_0$ of the joint probability $q(x_{t-1}, x_t \mid \tilde{x}_0)$ times the denoiser probability $\tilde{p}_\theta(\tilde{x}_0 \mid x_t)$.

An earlier token must fit both the clean token and the noise we observed.

In D3PM, we can choose rate matrices beyond the π family

In D3PM, we can choose rate matrices beyond the π family

We can build the rate matrix to reflect the structure of the data.

In D3PM, we can choose rate matrices beyond the π family

We can build the rate matrix to reflect the structure of the data.

For pixel intensities, we use a discretized Gaussian kernel, so nearby intensities are more likely replacements.

In D3PM, we can choose rate matrices beyond the π family

We can build the rate matrix to reflect the structure of the data.

For pixel intensities, we use a discretized Gaussian kernel, so nearby intensities are more likely replacements.

For text, we build a nearest-neighbor graph A from word embeddings and exponentiate its rate matrix:

$$Q_t = \exp\big(\tau_t\, R\big), \qquad R(i,j) = A_{ij} \;\; (i \neq j), \qquad R(i,i) = -\sum_{l \neq i} A_{il}$$

In D3PM, we can choose rate matrices beyond the π family

We can build the rate matrix to reflect the structure of the data.

For pixel intensities, we use a discretized Gaussian kernel, so nearby intensities are more likely replacements.

For text, we build a nearest-neighbor graph A from word embeddings and exponentiate its rate matrix:

$$Q_t = \exp\big(\tau_t\, R\big), \qquad R(i,j) = A_{ij} \;\; (i \neq j), \qquad R(i,i) = -\sum_{l \neq i} A_{il}$$

The transition matrix Q_t equals the matrix exponential of τ_t times the rate matrix R, whose off-diagonal entries R(i, j) are the graph weights A_{ij} and whose diagonal entries R(i, i) are negative the sum of A_{il} over every other token l.

In D3PM, we can choose rate matrices beyond the π family

We can build the rate matrix to reflect the structure of the data.

For pixel intensities, we use a discretized Gaussian kernel, so nearby intensities are more likely replacements.

For text, we build a nearest-neighbor graph A from word embeddings and exponentiate its rate matrix:

$$Q_t = \exp\big(\tau_t\, R\big), \qquad R(i,j) = A_{ij} \;\; (i \neq j), \qquad R(i,i) = -\sum_{l \neq i} A_{il}$$

The transition matrix Q_t equals the matrix exponential of τ_t times the rate matrix R, whose off-diagonal entries R(i, j) are the graph weights A_{ij} and whose diagonal entries R(i, i) are negative the sum of A_{il} over every other token l.

Because A is symmetric, this rate matrix satisfies detailed balance with the uniform distribution, so every fact from our framework still applies.

D3PM: which step changed the base?

Toy alphabet A and G. Each forward step keeps the base with probability 0.8 and flips it with probability 0.2. Start at A and observe G after two steps.

D3PM: which step changed the base?

Toy alphabet A and G. Each forward step keeps the base with probability 0.8 and flips it with probability 0.2. Start at A and observe G after two steps.

$$q(X_2 = \mathrm{G} \mid X_0 = \mathrm{A}) = (0.8)(0.2) + (0.2)(0.8) = 0.32$$

D3PM: which step changed the base?

Toy alphabet A and G. Each forward step keeps the base with probability 0.8 and flips it with probability 0.2. Start at A and observe G after two steps.

$$q(X_2 = \mathrm{G} \mid X_0 = \mathrm{A}) = (0.8)(0.2) + (0.2)(0.8) = 0.32$$

The final G can come from a flip in the first step or a flip in the second step.

D3PM: which step changed the base?

Toy alphabet A and G. Each forward step keeps the base with probability 0.8 and flips it with probability 0.2. Start at A and observe G after two steps.

$$q(X_2 = \mathrm{G} \mid X_0 = \mathrm{A}) = (0.8)(0.2) + (0.2)(0.8) = 0.32$$

The final G can come from a flip in the first step or a flip in the second step.

$$q(X_1 = \mathrm{A} \mid X_2 = \mathrm{G}, X_0 = \mathrm{A}) = \frac{(0.8)(0.2)}{0.32} = 0.5$$

D3PM: which step changed the base?

Toy alphabet A and G. Each forward step keeps the base with probability 0.8 and flips it with probability 0.2. Start at A and observe G after two steps.

$$q(X_2 = \mathrm{G} \mid X_0 = \mathrm{A}) = (0.8)(0.2) + (0.2)(0.8) = 0.32$$

The final G can come from a flip in the first step or a flip in the second step.

$$q(X_1 = \mathrm{A} \mid X_2 = \mathrm{G}, X_0 = \mathrm{A}) = \frac{(0.8)(0.2)}{0.32} = 0.5$$

Bayes gives the intermediate A probability 0.5. The intermediate G also has probability 0.5.

D3PM: which step changed the base?

Toy alphabet A and G. Each forward step keeps the base with probability 0.8 and flips it with probability 0.2. Start at A and observe G after two steps.

$$q(X_2 = \mathrm{G} \mid X_0 = \mathrm{A}) = (0.8)(0.2) + (0.2)(0.8) = 0.32$$

The final G can come from a flip in the first step or a flip in the second step.

$$q(X_1 = \mathrm{A} \mid X_2 = \mathrm{G}, X_0 = \mathrm{A}) = \frac{(0.8)(0.2)}{0.32} = 0.5$$

Bayes gives the intermediate A probability 0.5. The intermediate G also has probability 0.5.

A reverse step distributes probability over the earlier states that could explain the observation.

In D3PM, we use one transition matrix per step, with the structure chosen for the data

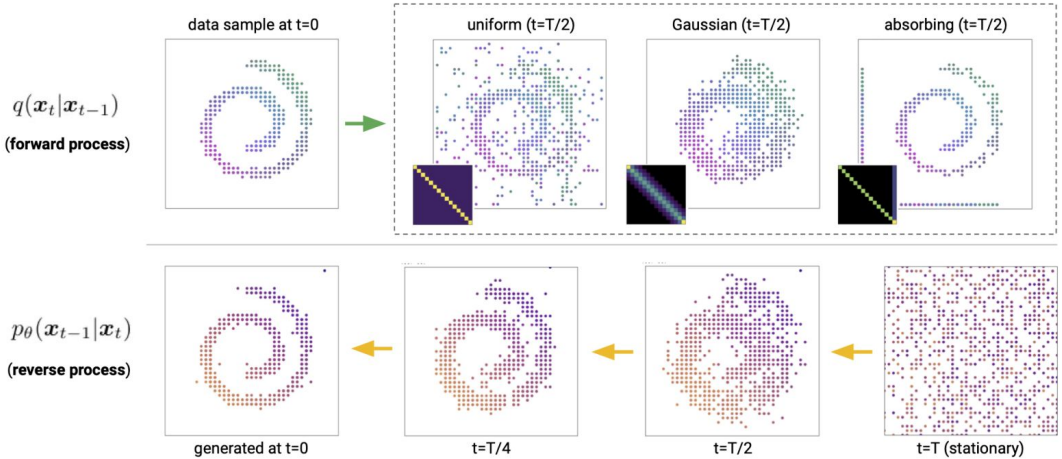


Figure 1: D3PM forward and (learned) reverse process applied to a quantized swiss roll. Each dot represents a 2D categorical variable. Top: samples from the uniform, discretized Gaussian, and absorbing state D3PM model forward processes, along with corresponding transition matrices $\boldsymbol{Q}$. Bottom: samples from a learned discretized Gaussian reverse process.

A Gaussian kernel on pixel intensities moves probability to nearby values first.

Austin,..,van den Berg. *NeurIPS* (2021)

Now look at the LM1B result in Table 2 of the paper

Table 2: Quantitative results on LM1B. Perplexity reported on the test set. Results are reported on two seeds. All models have context window length 128 and 12 layers unless otherwise noted. †Transformer XL is a 24 layer transformer. ‡rounded for readability, see Appendix B.2.2.

Metric:	Perplexity (↓)			Sample time‡ (s) (↓)		
inference steps:	1000	128	64	1000	128	64
D3PM uniform	137.9 ± 2.1	139.2 ± 1.2	145.0 ± 1.2	1.82	0.21	0.08
D3PM NN	149.5 ± 1.3	158.6 ± 2.2	160.4 ± 1.2	21.29	6.69	5.88
D3PM absorbing	76.9 ± 2.3	80.1 ± 1.2	83.6 ± 6.1	1.90	0.19	0.10
Transformer (ours)	-	43.6	-	-	0.26	-
Transformer XL [9]†	-	21.8	-	-	-	-

Austin,..,van den Berg. *NeurIPS* (2021)

On LM1B, uniform corruption has a perplexity bound of 137.9

Table 2: Quantitative results on LM1B. Perplexity reported on the test set. Results are reported on two seeds. All models have context window length 128 and 12 layers unless otherwise noted. †Transformer XL is a 24 layer transformer. ‡rounded for readability, see Appendix B.2.2.

Metric:	Perplexity (↓)			Sample time‡ (s) (↓)		
inference steps:	1000	128	64	1000	128	64
D3PM uniform	137.9 ± 2.1	139.2 ± 1.2	145.0 ± 1.2	1.82	0.21	0.08
D3PM NN	149.5 ± 1.3	158.6 ± 2.2	160.4 ± 1.2	21.29	6.69	5.88
D3PM absorbing	76.9 ± 2.3	80.1 ± 1.2	83.6 ± 6.1	1.90	0.19	0.10
Transformer (ours)	-	43.6	-	-	0.26	-
Transformer XL [9]†	-	21.8	-	-	-	-

Austin,..,van den Berg. *NeurIPS* (2021)

Absorbing corruption, which is masking, reaches 76.9 with the same model

Table 2: Quantitative results on LM1B. Perplexity reported on the test set. Results are reported on two seeds. All models have context window length 128 and 12 layers unless otherwise noted. †Transformer XL is a 24 layer transformer. ‡rounded for readability, see Appendix B.2.2.

Metric:	Perplexity (↓)			Sample time‡ (s) (↓)		
inference steps:	1000	128	64	1000	128	64
D3PM uniform	137.9 ± 2.1	139.2 ± 1.2	145.0 ± 1.2	1.82	0.21	0.08
D3PM NN	149.5 ± 1.3	158.6 ± 2.2	160.4 ± 1.2	21.29	6.69	5.88
D3PM absorbing	76.9 ± 2.3	80.1 ± 1.2	83.6 ± 6.1	1.90	0.19	0.10
Transformer (ours)	-	43.6	-	-	0.26	-
Transformer XL [9]†	-	21.8	-	-	-	-

Austin,..,van den Berg. *NeurIPS* (2021)

On images, the Gaussian kernel did better than uniform and absorbing corruption

Table 2: Quantitative results on LM1B. Perplexity reported on the test set. Results are reported on two seeds. All models have context window length 128 and 12 layers unless otherwise noted. †Transformer XL is a 24 layer transformer. ‡rounded for readability, see Appendix B.2.2.

Metric:	Perplexity (↓)			Sample time‡ (s) (↓)		
inference steps:	1000	128	64	1000	128	64
D3PM uniform	137.9 ± 2.1	139.2 ± 1.2	145.0 ± 1.2	1.82	0.21	0.08
D3PM NN	149.5 ± 1.3	158.6 ± 2.2	160.4 ± 1.2	21.29	6.69	5.88
D3PM absorbing	76.9 ± 2.3	80.1 ± 1.2	83.6 ± 6.1	1.90	0.19	0.10
Transformer (ours)	-	43.6	-	-	0.26	-
Transformer XL [9]†	-	21.8	-	-	-	-

Austin,..,van den Berg. *NeurIPS* (2021)

We encode the structure of the data in the rate matrix

Table 2: Quantitative results on LM1B. Perplexity reported on the test set. Results are reported on two seeds. All models have context window length 128 and 12 layers unless otherwise noted. †Transformer XL is a 24 layer transformer. ‡rounded for readability, see Appendix B.2.2.

Metric:	Perplexity (↓)			Sample time‡ (s) (↓)		
inference steps:	1000	128	64	1000	128	64
D3PM uniform	137.9 ± 2.1	139.2 ± 1.2	145.0 ± 1.2	1.82	0.21	0.08
D3PM NN	149.5 ± 1.3	158.6 ± 2.2	160.4 ± 1.2	21.29	6.69	5.88
D3PM absorbing	76.9 ± 2.3	80.1 ± 1.2	83.6 ± 6.1	1.90	0.19	0.10
Transformer (ours)	-	43.6	-	-	0.26	-
Transformer XL [9]†	-	21.8	-	-	-	-

Austin,..,van den Berg. *NeurIPS* (2021)

What if corrupted positions still looked like ordinary tokens, so we could revise any position?

UDLM

SIMPLE GUIDANCE MECHANISMS FOR DISCRETE DIFFUSION MODELS

Yair Schiff[*†], **Subham Sekhar Sahoo**[†], **Hao Phung**[†], **Guanghan Wang**[†],
Alexander Rush, & Volodymyr Kuleshov
Department of Computer Science, Cornell University
{yzs2,sss284,gw354,htp26,amr459,vk379}@cornell.edu

Hugo Dalla-Torre, Sam Boshar, Bernardo P. de Almeida, & Thomas Pierrot
InstaDeep
{h.dalla-torre,s.boshar,b.dealmeida,t.pierrot}@instadeep.com

Schiff,..,Kuleshov. *ICLR* (2025)

Now let π be uniform over the K tokens

Now let π be uniform over the K tokens

With uniform π, a noisy token may be the original or a replacement, and nothing marks which positions were corrupted.

Now let π be uniform over the K tokens

With uniform π, a noisy token may be the original or a replacement, and nothing marks which positions were corrupted.

The conditional reverse rate from the current token c to a ≠ c is

$$\hat{R}_t(c \to a \mid x_0) = \frac{\beta(t)}{K} \cdot \frac{\alpha_t \, \delta(a, x_0) + \frac{1-\alpha_t}{K}}{\alpha_t \, \delta(c, x_0) + \frac{1-\alpha_t}{K}}$$

Now let π be uniform over the K tokens

With uniform π, a noisy token may be the original or a replacement, and nothing marks which positions were corrupted.

The conditional reverse rate from the current token c to a ≠ c is

$$\hat{R}_t(c \to a \mid x_0) = \frac{\beta(t)}{K} \cdot \frac{\alpha_t\, \delta(a, x_0) + \frac{1-\alpha_t}{K}}{\alpha_t\, \delta(c, x_0) + \frac{1-\alpha_t}{K}}$$

The conditional reverse rate $\hat{R}_t(c \to a \mid x_0)$ from the current token c to a, given the clean token x_0, equals the forward rate β(t) divided by the vocabulary size K, times $\alpha_t \delta(a, x_0)$ plus one minus α_t over K, divided by $\alpha_t \delta(c, x_0)$ plus one minus α_t over K, where α_t is the survival probability and $\delta(a, x_0)$ is one when a equals x_0.

Now let π be uniform over the K tokens

With uniform π, a noisy token may be the original or a replacement, and nothing marks which positions were corrupted.

The conditional reverse rate from the current token c to a ≠ c is

$$\hat{R}_t(c \to a \mid x_0) = \frac{\beta(t)}{K} \cdot \frac{\alpha_t\,\delta(a, x_0) + \frac{1-\alpha_t}{K}}{\alpha_t\,\delta(c, x_0) + \frac{1-\alpha_t}{K}}$$

The conditional reverse rate $\hat{R}_t(c \to a \mid x_0)$ from the current token c to a, given the clean token x_0, equals the forward rate β(t) divided by the vocabulary size K, times $\alpha_t\delta(a, x_0)$ plus one minus α_t over K, divided by $\alpha_t\delta(c, x_0)$ plus one minus α_t over K, where α_t is the survival probability and $\delta(a, x_0)$ is one when a equals x_0.

A token the model already placed can still be revised, which cannot happen with masking.

Divide the conditional reverse rate by the noise rate $\beta(t)/K$

Divide the conditional reverse rate by the noise rate β(t)/K

$$\frac{\hat{R}_t(c \to a \mid x_0)}{\beta(t)/K} = \begin{cases} 1 + \frac{K\alpha_t}{1-\alpha_t} & a = x_0 \\ \left(1 + \frac{K\alpha_t}{1-\alpha_t}\right)^{-1} & c = x_0 \\ 1 & \text{otherwise} \end{cases}$$

Divide the conditional reverse rate by the noise rate β(t)/K

$$\frac{\hat{R}_t(c \to a \mid x_0)}{\beta(t)/K} = \begin{cases} 1 + \frac{K\alpha_t}{1-\alpha_t} & a = x_0 \\ \left(1 + \frac{K\alpha_t}{1-\alpha_t}\right)^{-1} & c = x_0 \\ 1 & \text{otherwise} \end{cases}$$

The conditional reverse rate $\hat{R}_t(c \to a \mid x_0)$ divided by the noise rate β(t)/K equals one plus $K\alpha_t$ divided by one minus α_t when a is the clean token x_0, the reciprocal of that factor when the current token c is x_0, and one otherwise.

Divide the conditional reverse rate by the noise rate β(t)/K

$$\frac{\hat{R}_t(c \to a \mid x_0)}{\beta(t)/K} = \begin{cases} 1 + \frac{K\alpha_t}{1-\alpha_t} & a = x_0 \\ \left(1 + \frac{K\alpha_t}{1-\alpha_t}\right)^{-1} & c = x_0 \\ 1 & \text{otherwise} \end{cases}$$

The conditional reverse rate $\hat{R}_t(c \to a \mid x_0)$ divided by the noise rate β(t)/K equals one plus $K\alpha_t$ divided by one minus α_t when a is the clean token x_0, the reciprocal of that factor when the current token c is x_0, and one otherwise.

With K = 4 and α_t = 1/2, the factor is 5. These are the rates 5, 0.2, and 1 from our example.

Let’s compute the rate error for uniform noise on one base

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Take clean A, current C, $\alpha_t = 1/2$, unit forward rates, and a denoiser that predicts (0.5, 0.3, 0.1, 0.1) for (A, C, G, T).

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Take clean A, current C, α_t = 1/2, unit forward rates, and a denoiser that predicts (0.5, 0.3, 0.1, 0.1) for (A, C, G, T).

Average the conditional rates with the denoiser:

$$\hat{R}^{\theta}(\mathrm{C} \to \mathrm{A}) = 0.5(5) + 0.3(0.2) + 0.1(1) + 0.1(1) = 2.76$$

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Take clean A, current C, α_t = 1/2, unit forward rates, and a denoiser that predicts (0.5, 0.3, 0.1, 0.1) for (A, C, G, T).

Average the conditional rates with the denoiser:

$$\hat{R}^{\theta}(\mathrm{C} \to \mathrm{A}) = 0.5(5) + 0.3(0.2) + 0.1(1) + 0.1(1) = 2.76$$

The model rate $\hat{R}^{\theta}(\mathrm{C} \to \mathrm{A})$ averages the conditional rates 5, 0.2, 1, and 1 with the denoiser probabilities 0.5, 0.3, 0.1, and 0.1, which is 2.76.

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Take clean A, current C, α_t = 1/2, unit forward rates, and a denoiser that predicts (0.5, 0.3, 0.1, 0.1) for (A, C, G, T).

Average the conditional rates with the denoiser:

$$\hat{R}^{\theta}(\mathrm{C} \to \mathrm{A}) = 0.5(5) + 0.3(0.2) + 0.1(1) + 0.1(1) = 2.76$$

The target rates are 5, 1, and 1, and the other two model rates are 1.16:

$$\ell(5,\ 2.76) + 2\,\ell(1,\ 1.16) = 0.731 + 2(0.012) = 0.754$$

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Take clean A, current C, α_t = 1/2, unit forward rates, and a denoiser that predicts (0.5, 0.3, 0.1, 0.1) for (A, C, G, T).

Average the conditional rates with the denoiser:

$$\hat{R}^{\theta}(\mathrm{C} \to \mathrm{A}) = 0.5(5) + 0.3(0.2) + 0.1(1) + 0.1(1) = 2.76$$

The target rates are 5, 1, and 1, and the other two model rates are 1.16:

$$\ell(5,\ 2.76) + 2\,\ell(1,\ 1.16) = 0.731 + 2(0.012) = 0.754$$

The rate error $\ell(5, 2.76)$ for the jump to A plus twice the rate error $\ell(1, 1.16)$ for the jumps to G and T equals 0.731 plus 2 times 0.012, which is 0.754.

Let’s compute the rate error for uniform noise on one base

Every token can jump to every other token, so all K – 1 destinations contribute to the rate error.

Take clean A, current C, α_t = 1/2, unit forward rates, and a denoiser that predicts (0.5, 0.3, 0.1, 0.1) for (A, C, G, T).

Average the conditional rates with the denoiser:

$$\hat{R}^{\theta}(\mathrm{C} \to \mathrm{A}) = 0.5(5) + 0.3(0.2) + 0.1(1) + 0.1(1) = 2.76$$

The target rates are 5, 1, and 1, and the other two model rates are 1.16:

$$\ell(5,\ 2.76) + 2\,\ell(1,\ 1.16) = 0.731 + 2(0.012) = 0.754$$

Most of the loss comes from underestimating the rate of returning to A.

UDLM: one update of a DNA base

Continue the previous example at current base C. The learned rates to A, G and T are 2.76, 1.16 and 1.16. Take a first-order step of length 0.05.

UDLM: one update of a DNA base

Continue the previous example at current base C. The learned rates to A, G and T are 2.76, 1.16 and 1.16. Take a first-order step of length 0.05.

$$R_{\text{out}} = 2.76 + 1.16 + 1.16 = 5.08, \qquad p(\text{stay}) = 1 - 0.05(5.08) = 0.746$$

UDLM: one update of a DNA base

Continue the previous example at current base C. The learned rates to A, G and T are 2.76, 1.16 and 1.16. Take a first-order step of length 0.05.

$$R_{\text{out}} = 2.76 + 1.16 + 1.16 = 5.08, \qquad p(\text{stay}) = 1 - 0.05(5.08) = 0.746$$

The total outgoing rate gives a 25.4% chance of a change in this step.

UDLM: one update of a DNA base

Continue the previous example at current base C. The learned rates to A, G and T are 2.76, 1.16 and 1.16. Take a first-order step of length 0.05.

$$R_{\text{out}} = 2.76 + 1.16 + 1.16 = 5.08, \qquad p(\text{stay}) = 1 - 0.05(5.08) = 0.746$$

The total outgoing rate gives a 25.4% chance of a change in this step.

$$p_{\text{step}}(\text{A}, \text{C}, \text{G}, \text{T}) = (0.138,\ 0.746,\ 0.058,\ 0.058)$$

UDLM: one update of a DNA base

Continue the previous example at current base C. The learned rates to A, G and T are 2.76, 1.16 and 1.16. Take a first-order step of length 0.05.

$$R_{\text{out}} = 2.76 + 1.16 + 1.16 = 5.08, \qquad p(\text{stay}) = 1 - 0.05(5.08) = 0.746$$

The total outgoing rate gives a 25.4% chance of a change in this step.

$$p_{\text{step}}(\text{A}, \text{C}, \text{G}, \text{T}) = (0.138,\ 0.746,\ 0.058,\ 0.058)$$

Multiply each destination rate by 0.05, then give C the remaining probability.

UDLM: one update of a DNA base

Continue the previous example at current base C. The learned rates to A, G and T are 2.76, 1.16 and 1.16. Take a first-order step of length 0.05.

$$R_{\text{out}} = 2.76 + 1.16 + 1.16 = 5.08, \qquad p(\text{stay}) = 1 - 0.05(5.08) = 0.746$$

The total outgoing rate gives a 25.4% chance of a change in this step.

$$p_{\text{step}}(\text{A}, \text{C}, \text{G}, \text{T}) = (0.138,\ 0.746,\ 0.058,\ 0.058)$$

Multiply each destination rate by 0.05, then give C the remaining probability.

The current letter can survive this step, and a later step can revise it again.

The paper has a closed-form bound for uniform noise, and guidance built on it

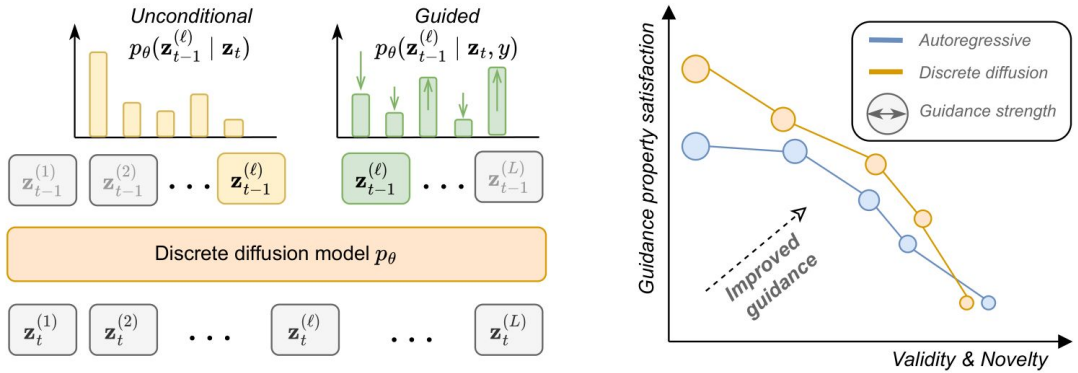


Figure 1: *(Left)* Adapting guidance to discrete diffusion. Models output a factorized discrete distribution for each denoised token. With our guidance mechanisms, we adjust these probabilities according to a guidance model – either a conditional diffusion model in classifier-free guidance (Section 3.1) or a separately trained classifier for classifier-based guidance (Section 3.2). *(Right)* Relative to autoregressive models, which make local predictions one token at a time, discrete diffusion models denoise the entire sequence at every iteration, allowing for more guidable outputs.

The bound is the rate loss above, written out for the uniform kernel.

Schiff,..,Kuleshov. *ICLR* (2025)

Here is Table 1 of the paper, with perplexity on six datasets

Table 1: UDLM performs best with smaller vocabs. Perplexity (↓) on various datasets. Best values are **bolded.** * indicates values reported from early stopping on the validation set; otherwise validation performance at the end of training is used. †From Sahoo et al. (2024a). $From Lou et al. (2023).

	|Vocab.|	AR	MDLM	UDLM
Species10	12	**2.88**	$3.17_{\leq}$	$3.15_{\leq}$
QM9*	40	2.19	$2.12_{\leq}$	$\mathbf{2.02}_{\leq}$
CIFAR10	256	-	$\mathbf{9.14}_{\leq}$	$11.21_{\leq}$
text8	35	**2.35**$	$2.62_{\leq}$	$2.71_{\leq}$
Amazon*	30,522	**21.67**	$24.93_{\leq}$	$27.27_{\leq}$
LM1B	30,522	**22.32**†	$27.04^{\dagger}_{\leq}$	$31.28_{\leq}$

Schiff,..,Kuleshov. *ICLR* (2025)

On Species10, with 12 tokens, UDLM is ahead of MDLM

Table 1: UDLM performs best with smaller vocabs. Perplexity (↓) on various datasets. Best values are **bolded.** * indicates values reported from early stopping on the validation set; otherwise validation performance at the end of training is used. †From Sahoo et al. (2024a). $From Lou et al. (2023).

	|Vocab.|	AR	MDLM	UDLM
Species10	12	**2.88**	$3.17_{\leq}$	$3.15_{\leq}$
QM9*	40	2.19	$2.12_{\leq}$	$\mathbf{2.02}_{\leq}$
CIFAR10	256	-	$\mathbf{9.14}_{\leq}$	$11.21_{\leq}$
text8	35	**2.35**$	$2.62_{\leq}$	$2.71_{\leq}$
Amazon*	30,522	**21.67**	$24.93_{\leq}$	$27.27_{\leq}$
LM1B	30,522	**22.32**†	$27.04^{\dagger}_{\leq}$	$31.28_{\leq}$

Schiff,..,Kuleshov. *ICLR* (2025)

On QM9, with 40 tokens, it is ahead again

Table 1: UDLM performs best with smaller vocabs. Perplexity (↓) on various datasets. Best values are **bolded.** * indicates values reported from early stopping on the validation set; otherwise validation performance at the end of training is used. †From Sahoo et al. (2024a). $From Lou et al. (2023).

	|Vocab.|	AR	MDLM	UDLM
Species10	12	**2.88**	3.17$_{\leq}$	3.15$_{\leq}$
QM9*	40	2.19	2.12$_{\leq}$	**2.02**$_{\leq}$
CIFAR10	256	-	**9.14**$_{\leq}$	11.21$_{\leq}$
text8	35	**2.35**$^{\$}$	2.62$_{\leq}$	2.71$_{\leq}$
Amazon*	30,522	**21.67**	24.93$_{\leq}$	27.27$_{\leq}$
LM1B	30,522	**22.32**†	27.04$^{\dagger}_{\leq}$	31.28$_{\leq}$

Schiff,..,Kuleshov. *ICLR* (2025)

On LM1B, with 30,522 tokens, masking is well ahead

Table 1: UDLM performs best with smaller vocabs. Perplexity (↓) on various datasets. Best values are **bolded.** * indicates values reported from early stopping on the validation set; otherwise validation performance at the end of training is used. †From Sahoo et al. (2024a). $From Lou et al. (2023).

	|Vocab.|	AR	MDLM	UDLM
Species10	12	**2.88**	3.17$_{\leq}$	3.15$_{\leq}$
QM9*	40	2.19	2.12$_{\leq}$	**2.02**$_{\leq}$
CIFAR10	256	-	**9.14**$_{\leq}$	11.21$_{\leq}$
text8	35	**2.35**$^{\$}$	2.62$_{\leq}$	2.71$_{\leq}$
Amazon*	30,522	**21.67**	24.93$_{\leq}$	27.27$_{\leq}$
LM1B	30,522	**22.32**†	27.04$^{\dagger}_{\leq}$	31.28$_{\leq}$

Schiff,..,Kuleshov. *ICLR* (2025)

With a larger vocabulary, a uniform replacement is harder to detect

Table 1: UDLM performs best with smaller vocabs. Perplexity (↓) on various datasets. Best values are **bolded.** * indicates values reported from early stopping on the validation set; otherwise validation performance at the end of training is used. †From Sahoo et al. (2024a). $From Lou et al. (2023).

	|Vocab.|	AR	MDLM	UDLM
Species10	12	**2.88**	$3.17_{\leq}$	$3.15_{\leq}$
QM9*	40	2.19	$2.12_{\leq}$	$\mathbf{2.02}_{\leq}$
CIFAR10	256	-	$\mathbf{9.14}_{\leq}$	$11.21_{\leq}$
text8	35	**2.35**$	$2.62_{\leq}$	$2.71_{\leq}$
Amazon*	30,522	**21.67**	$24.93_{\leq}$	$27.27_{\leq}$
LM1B	30,522	**22.32**†	$27.04^{\dagger}_{\leq}$	$31.28_{\leq}$

Schiff,..,Kuleshov. *ICLR* (2025)

If generation only needs probability ratios, why not predict those ratios directly?

SEDD: Score Entropy Discrete Diffusion Models

Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution

Aaron Lou [1] **Chenlin Meng** [1 2] **Stefano Ermon** [1]

Lou,..,Ermon. *ICML* (2024)

Next, we predict the ratios directly, as in SEDD

Next, we predict the ratios directly, as in SEDD

For our reverse rate, we need the ratio $p_t(z^{i \leftarrow a})/p_t(z)$, and we have been reaching it through a denoiser.

Next, we predict the ratios directly, as in SEDD

For our reverse rate, we need the ratio $p_t(z^{i \leftarrow a})/p_t(z)$, and we have been reaching it through a denoiser.

In SEDD, we predict that ratio with the network, and call it the concrete score $s_\theta(z)_{i,a}$.

Next, we predict the ratios directly, as in SEDD

For our reverse rate, we need the ratio $p_t(z^{i \leftarrow a})/p_t(z)$, and we have been reaching it through a denoiser.

In SEDD, we predict that ratio with the network, and call it the concrete score $s_\theta(z)_{i,a}$.

With forward rate λ, target ratio r, and predicted ratio s, the rate error becomes score entropy:

$$\ell(\lambda r,\ \lambda s) = \lambda \left[s - r \log s + r(\log r - 1) \right]$$

Next, we predict the ratios directly, as in SEDD

For our reverse rate, we need the ratio $p_t(z^{i \leftarrow a})/p_t(z)$, and we have been reaching it through a denoiser.

In SEDD, we predict that ratio with the network, and call it the concrete score $s_\theta(z)_{i,a}$.

With forward rate λ, target ratio r, and predicted ratio s, the rate error becomes score entropy:

$$\ell(\lambda r,\ \lambda s) = \lambda \left[s - r \log s + r(\log r - 1) \right]$$

The rate error ℓ(λr, λs) between the target rate λr and the model rate λs equals the forward rate λ times the predicted ratio s, minus the target ratio r times log s, plus r times the quantity log r minus one.

Next, we predict the ratios directly, as in SEDD

For our reverse rate, we need the ratio $p_t(z^{i \leftarrow a})/p_t(z)$, and we have been reaching it through a denoiser.

In SEDD, we predict that ratio with the network, and call it the concrete score $s_\theta(z)_{i,a}$.

With forward rate λ, target ratio r, and predicted ratio s, the rate error becomes score entropy:

$$\ell(\lambda r,\ \lambda s) = \lambda \left[s - r \log s + r(\log r - 1) \right]$$

The rate error ℓ(λr, λs) between the target rate λr and the model rate λs equals the forward rate λ times the predicted ratio s, minus the target ratio r times log s, plus r times the quantity log r minus one.

The bound and its minimizer are the same as ours. Only the parameterization changes.

Next, we predict the ratios directly, as in SEDD

For our reverse rate, we need the ratio $p_t(z^{i \leftarrow a})/p_t(z)$, and we have been reaching it through a denoiser.

In SEDD, we predict that ratio with the network, and call it the concrete score $s_\theta(z)_{i,a}$.

With forward rate λ, target ratio r, and predicted ratio s, the rate error becomes score entropy:

$$\ell(\lambda r,\ \lambda s) = \lambda \left[s - r \log s + r(\log r - 1) \right]$$

The rate error ℓ(λr, λs) between the target rate λr and the model rate λs equals the forward rate λ times the predicted ratio s, minus the target ratio r times log s, plus r times the quantity log r minus one.

Doubling a predicted ratio doubles that model jump rate; score entropy penalizes the resulting rate error.

SEDD: a ratio becomes a reverse rate

Toy noisy-state probabilities: ACGT has mass 0.04 and AGGT has mass 0.12. The forward rate from AGGT to ACGT is 2. The network predicts a ratio of 2.

SEDD: a ratio becomes a reverse rate

Toy noisy-state probabilities: ACGT has mass 0.04 and AGGT has mass 0.12. The forward rate from AGGT to ACGT is 2. The network predicts a ratio of 2.

$$r = \frac{0.12}{0.04} = 3, \qquad R_{\text{reverse}} = 2(3) = 6, \qquad R_\theta = 2(2) = 4$$

SEDD: a ratio becomes a reverse rate

Toy noisy-state probabilities: ACGT has mass 0.04 and AGGT has mass 0.12. The forward rate from AGGT to ACGT is 2. The network predicts a ratio of 2.

$$r = \frac{0.12}{0.04} = 3, \qquad R_{\text{reverse}} = 2(3) = 6, \qquad R_\theta = 2(2) = 4$$

The neighbor is three times as likely, so the exact reverse rate is 6. The model currently gives 4.

SEDD: a ratio becomes a reverse rate

Toy noisy-state probabilities: ACGT has mass 0.04 and AGGT has mass 0.12. The forward rate from AGGT to ACGT is 2. The network predicts a ratio of 2.

$$r = \frac{0.12}{0.04} = 3, \qquad R_{\text{reverse}} = 2(3) = 6, \qquad R_\theta = 2(2) = 4$$

The neighbor is three times as likely, so the exact reverse rate is 6. The model currently gives 4.

$$\ell(6, 4) = 6 \log\left(\frac{6}{4}\right) - 6 + 4 \approx 0.433$$

SEDD: a ratio becomes a reverse rate

Toy noisy-state probabilities: ACGT has mass 0.04 and AGGT has mass 0.12. The forward rate from AGGT to ACGT is 2. The network predicts a ratio of 2.

$$r = \frac{0.12}{0.04} = 3, \qquad R_{\text{reverse}} = 2(3) = 6, \qquad R_\theta = 2(2) = 4$$

The neighbor is three times as likely, so the exact reverse rate is 6. The model currently gives 4.

$$\ell(6, 4) = 6 \log\left(\frac{6}{4}\right) - 6 + 4 \approx 0.433$$

This edge contributes 0.433 to the rate error. The contribution reaches zero when the predicted ratio is 3.

SEDD: a ratio becomes a reverse rate

Toy noisy-state probabilities: ACGT has mass 0.04 and AGGT has mass 0.12. The forward rate from AGGT to ACGT is 2. The network predicts a ratio of 2.

$$r = \frac{0.12}{0.04} = 3, \qquad R_{\text{reverse}} = 2(3) = 6, \qquad R_\theta = 2(2) = 4$$

The neighbor is three times as likely, so the exact reverse rate is 6. The model currently gives 4.

$$\ell(6, 4) = 6 \log\left(\frac{6}{4}\right) - 6 + 4 \approx 0.433$$

This edge contributes 0.433 to the rate error. The contribution reaches zero when the predicted ratio is 3.

The score tells us how strongly probability should flow toward a neighboring sequence.

Score entropy is our rate loss, written for the ratios

Discrete Diffusion Modeling by Estimating the Ratios of the Data Distribution

Aaron Lou [1] **Chenlin Meng** [1 2] **Stefano Ermon** [1]

Lou,..,Ermon. *ICML* (2024)

Now look at the zero-shot perplexity in Table 1 of the paper

Size	Model	LAMBADA	WikiText2	PTB	WikiText103	1BW
Small	GPT-2	**45.04**	42.43	138.43	41.60	**75.20**
	SEDD Absorb	≤50.92	≤**41.84**	≤**114.24**	≤**40.62**	≤79.29
	SEDD Uniform	≤65.40	≤50.27	≤140.12	≤49.60	≤101.37
	D3PM	≤93.47	≤77.28	≤200.82	≤75.16	≤138.92
	PLAID	≤57.28	≤51.80	≤142.60	≤50.86	≤91.12
Medium	GPT-2	**35.66**	31.80	123.14	31.39	**55.72**
	SEDD Absorb	≤42.77	≤**31.04**	≤**87.12**	≤**29.98**	≤61.19
	SEDD Uniform	≤51.28	≤38.93	≤102.28	≤36.81	≤79.12

Table 1: **Zero-shot unconditional perplexity (↓) on a variety of datasets.** For a fixed size, the best perplexity is **bolded**. Our SEDD model with absorbing transition beats GPT-2 (Radford et al., 2019) on a majority of the tasks and entirely outperforms prior language diffusion models (Austin et al., 2021; Gulrajani & Hashimoto, 2023).

Lou,..,Ermon. *ICML* (2024)

GPT-2 small is the baseline on five datasets

Size	Model	LAMBADA	WikiText2	PTB	WikiText103	1BW
Small	GPT-2	**45.04**	42.43	138.43	41.60	**75.20**
	SEDD Absorb	≤50.92	≤**41.84**	≤**114.24**	≤**40.62**	≤79.29
	SEDD Uniform	≤65.40	≤50.27	≤140.12	≤49.60	≤101.37
	D3PM	≤93.47	≤77.28	≤200.82	≤75.16	≤138.92
	PLAID	≤57.28	≤51.80	≤142.60	≤50.86	≤91.12
Medium	GPT-2	**35.66**	31.80	123.14	31.39	**55.72**
	SEDD Absorb	≤42.77	≤**31.04**	≤**87.12**	≤**29.98**	≤61.19
	SEDD Uniform	≤51.28	≤38.93	≤102.28	≤36.81	≤79.12

Table 1: **Zero-shot unconditional perplexity (↓) on a variety of datasets.** For a fixed size, the best perplexity is **bolded**. Our SEDD model with absorbing transition beats GPT-2 (Radford et al., 2019) on a majority of the tasks and entirely outperforms prior language diffusion models (Austin et al., 2021; Gulrajani & Hashimoto, 2023).

Lou,..,Ermon. *ICML* (2024)

SEDD with absorbing noise is better on three of them

Size	Model	LAMBADA	WikiText2	PTB	WikiText103	1BW
Small	GPT-2	**45.04**	42.43	138.43	41.60	**75.20**
	SEDD Absorb	≤50.92	≤**41.84**	≤**114.24**	≤**40.62**	≤79.29
	SEDD Uniform	≤65.40	≤50.27	≤140.12	≤49.60	≤101.37
	D3PM	≤93.47	≤77.28	≤200.82	≤75.16	≤138.92
	PLAID	≤57.28	≤51.80	≤142.60	≤50.86	≤91.12
Medium	GPT-2	**35.66**	31.80	123.14	31.39	**55.72**
	SEDD Absorb	≤42.77	≤**31.04**	≤**87.12**	≤**29.98**	≤61.19
	SEDD Uniform	≤51.28	≤38.93	≤102.28	≤36.81	≤79.12

Table 1: **Zero-shot unconditional perplexity (↓) on a variety of datasets.** For a fixed size, the best perplexity is **bolded**. Our SEDD model with absorbing transition beats GPT-2 (Radford et al., 2019) on a majority of the tasks and entirely outperforms prior language diffusion models (Austin et al., 2021; Gulrajani & Hashimoto, 2023).

Lou,..,Ermon. *ICML* (2024)

Uniform noise is behind absorbing noise on every dataset here

Size	Model	LAMBADA	WikiText2	PTB	WikiText103	1BW
Small	GPT-2	**45.04**	42.43	138.43	41.60	**75.20**
	SEDD Absorb	≤50.92	≤**41.84**	≤**114.24**	≤**40.62**	≤79.29
	SEDD Uniform	≤65.40	≤50.27	≤140.12	≤49.60	≤101.37
	D3PM	≤93.47	≤77.28	≤200.82	≤75.16	≤138.92
	PLAID	≤57.28	≤51.80	≤142.60	≤50.86	≤91.12
Medium	GPT-2	**35.66**	31.80	123.14	31.39	**55.72**
	SEDD Absorb	≤42.77	≤**31.04**	≤**87.12**	≤**29.98**	≤61.19
	SEDD Uniform	≤51.28	≤38.93	≤102.28	≤36.81	≤79.12

Table 1: **Zero-shot unconditional perplexity (↓) on a variety of datasets.** For a fixed size, the best perplexity is **bolded**. Our SEDD model with absorbing transition beats GPT-2 (Radford et al., 2019) on a majority of the tasks and entirely outperforms prior language diffusion models (Austin et al., 2021; Gulrajani & Hashimoto, 2023).

Lou,..,Ermon. *ICML* (2024)

With the ratio parameterization, the gap to autoregressive models closed

Size	Model	LAMBADA	WikiText2	PTB	WikiText103	1BW
Small	GPT-2	**45.04**	42.43	138.43	41.60	**75.20**
	SEDD Absorb	≤50.92	≤**41.84**	≤**114.24**	≤**40.62**	≤79.29
	SEDD Uniform	≤65.40	≤50.27	≤140.12	≤49.60	≤101.37
	D3PM	≤93.47	≤77.28	≤200.82	≤75.16	≤138.92
	PLAID	≤57.28	≤51.80	≤142.60	≤50.86	≤91.12
Medium	GPT-2	**35.66**	31.80	123.14	31.39	**55.72**
	SEDD Absorb	≤42.77	≤**31.04**	≤**87.12**	≤**29.98**	≤61.19
	SEDD Uniform	≤51.28	≤38.93	≤102.28	≤36.81	≤79.12

Table 1: **Zero-shot unconditional perplexity (↓) on a variety of datasets.** For a fixed size, the best perplexity is **bolded**. Our SEDD model with absorbing transition beats GPT-2 (Radford et al., 2019) on a majority of the tasks and entirely outperforms prior language diffusion models (Austin et al., 2021; Gulrajani & Hashimoto, 2023).

Lou,..,Ermon. *ICML* (2024)

Since autoregression is so strong, could we combine it with masked diffusion?

BD3-LMs: Block Diffusion Language Models

BLOCK DIFFUSION: INTERPOLATING BETWEEN AUTOREGRESSIVE AND DIFFUSION LANGUAGE MODELS

Marianne Arriola[†] [*] **Aaron Kerem Gokaslan**[†] **Justin T. Chiu**[‡] **Zhihan Yang**[†]

Zhixuan Qi[†] **Jiaqi Han**[¶] **Subham Sekhar Sahoo**[†] **Volodymyr Kuleshov**[†]

Arriola,..,Kuleshov. *ICLR* (2025)

Next, we combine masked diffusion with autoregression

Next, we combine masked diffusion with autoregression

With masked diffusion, we generate every position in parallel, and with autoregression, we get exact likelihoods and arbitrary length.

Next, we combine masked diffusion with autoregression

With masked diffusion, we generate every position in parallel, and with autoregression, we get exact likelihoods and arbitrary length.

In block diffusion, we split a sequence into B blocks of length L′ and keep the chain rule between them:

Next, we combine masked diffusion with autoregression

With masked diffusion, we generate every position in parallel, and with autoregression, we get exact likelihoods and arbitrary length.

In block diffusion, we split a sequence into B blocks of length L′ and keep the chain rule between them:

$$\log p_\theta(x) = \sum_{b=1}^{B} \log p_\theta\left(x^b \mid x^{<b}\right)$$

Next, we combine masked diffusion with autoregression

With masked diffusion, we generate every position in parallel, and with autoregression, we get exact likelihoods and arbitrary length.

In block diffusion, we split a sequence into B blocks of length L′ and keep the chain rule between them:

$$\log p_\theta(x) = \sum_{b=1}^{B} \log p_\theta\left(x^b \mid x^{<b}\right)$$

The log-likelihood log $p_\theta(x)$ of a sequence x equals the sum over blocks b from 1 to B of log $p_\theta(x^b \mid x^{<b})$, the log probability of block x^b given every earlier block $x^{<b}$.

Next, we combine masked diffusion with autoregression

With masked diffusion, we generate every position in parallel, and with autoregression, we get exact likelihoods and arbitrary length.

In block diffusion, we split a sequence into B blocks of length L′ and keep the chain rule between them:

$$\log p_\theta(x) = \sum_{b=1}^{B} \log p_\theta\left(x^b \mid x^{<b}\right)$$

The log-likelihood $\log p_\theta(x)$ of a sequence x equals the sum over blocks b from 1 to B of $\log p_\theta(x^b \mid x^{<b})$, the log probability of block x^b given every earlier block $x^{<b}$.

Each factor is a distribution over one block given a clean prefix, so we can model it with masked diffusion.

Now we need a loss for each block

Now we need a loss for each block

Bound each factor with the rate loss:

$$-\log p_\theta(x) \le \sum_{b=1}^{B} \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t} \mathbb{E}\Big[\sum_{i \in b:\, Z_t^i = m} -\log d_\theta^i\big(x^i \mid Z_t^b,\ x^{<b}\big) \Big] dt$$

Now we need a loss for each block

Bound each factor with the rate loss:

$$-\log p_\theta(x) \le \sum_{b=1}^{B} \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}\Big[\sum_{i \in b:\, Z_t^i = m} -\log d_\theta^i\big(x^i \mid Z_t^b,\ x^{<b}\big) \Big] dt$$

The negative log-likelihood of x is at most the sum over blocks b of the integral over time of negative $\dot{\alpha}_t$ divided by one minus α_t, times the expected sum over masked positions i in block b of negative log $d^i_\theta(x^i \mid Z^b_t, x^{<b})$, where we give the denoiser the noisy block Z^b_t and the clean earlier blocks $x^{<b}$.

Now we need a loss for each block

Bound each factor with the rate loss:

$$-\log p_\theta(x) \le \sum_{b=1}^{B} \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}\Big[\sum_{i \in b:\, Z_t^i = m} -\log d_\theta^i\big(x^i \mid Z_t^b,\, x^{<b}\big) \Big] dt$$

The negative log-likelihood of x is at most the sum over blocks b of the integral over time of negative $\dot{\alpha}_t$ divided by one minus α_t, times the expected sum over masked positions i in block b of negative log $d^i_\theta(x^i \mid Z^b_t, x^{<b})$, where we give the denoiser the noisy block Z^b_t and the clean earlier blocks $x^{<b}$.

The rate matrix acts only on the current block, and the prefix is a fixed condition, so all five facts hold block by block.

Now we need a loss for each block

Bound each factor with the rate loss:

$$-\log p_\theta(x) \le \sum_{b=1}^{B} \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}\Big[\sum_{i \in b:\, Z_t^i = m} -\log d_\theta^i\big(x^i \mid Z_t^b,\, x^{<b}\big) \Big] dt$$

The negative log-likelihood of x is at most the sum over blocks b of the integral over time of negative $\dot{\alpha}_t$ divided by one minus α_t, times the expected sum over masked positions i in block b of negative log $d^i_\theta(x^i \mid Z^b_t, x^{<b})$, where we give the denoiser the noisy block Z^b_t and the clean earlier blocks $x^{<b}$.

Each finished block supplies context for the next, while positions inside the current block are filled together.

So each block has the MDLM loss, given the clean prefix

Bound each factor with the rate loss:

$$-\log p_\theta(x) \le \sum_{b=1}^{B} \int_0^1 \frac{-\dot{\alpha}_t}{1-\alpha_t}\, \mathbb{E}\Big[\sum_{i \in b:\, Z_t^i = m} -\log d_\theta^i\big(x^i \mid Z_t^b,\, x^{<b}\big) \Big] dt$$

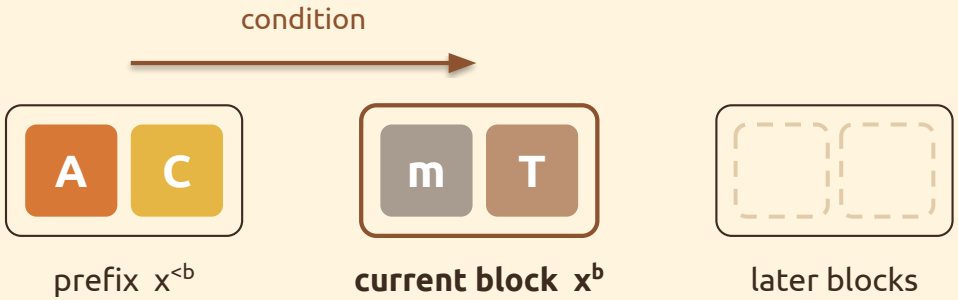
The negative log-likelihood of x is at most the sum over blocks b of the integral over time of negative $\dot{\alpha}_t$ divided by one minus α_t, times the expected sum over masked positions i in block b of negative log $d^i_\theta(x^i \mid Z^b_t, x^{<b})$, where we give the denoiser the noisy block Z^b_t and the clean earlier blocks $x^{<b}$.

The rate matrix acts only on the current block, and the prefix is a fixed condition, so all five facts hold block by block.

With a block size of L′ = 1 we recover autoregression, and with L′ = L we recover MDLM.

Let’s generate ACGTAC in blocks of two

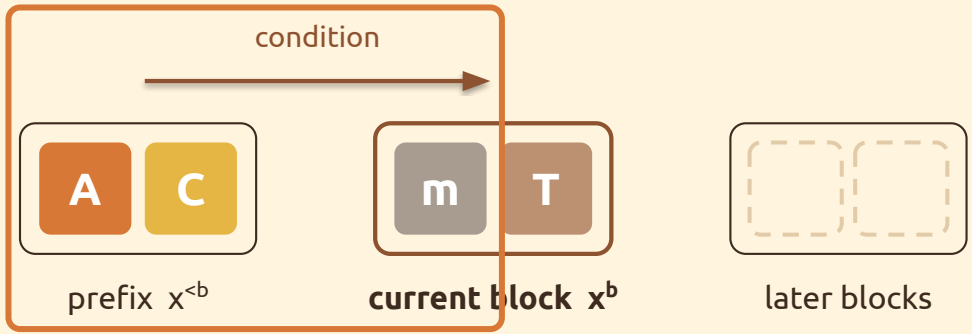
$$p_\theta(x) = \prod_b p_\theta(x^b \mid x^{<b})$$



block size L′ = 2: generate AC, then GT given AC, then AC given ACGT

The first block is already clean, and it conditions everything that follows

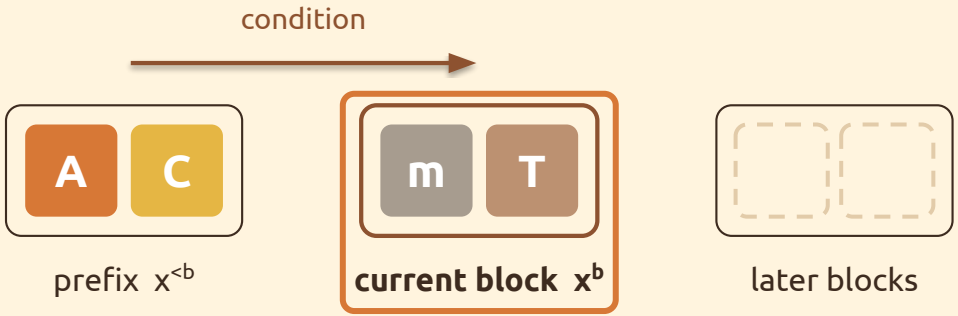
$$p_\theta(x) = \prod_b p_\theta(x^b \mid x^{<b})$$



block size L′ = 2: generate AC, then GT given AC, then AC given ACGT

Inside the current block, we reveal tokens in any order by masked diffusion

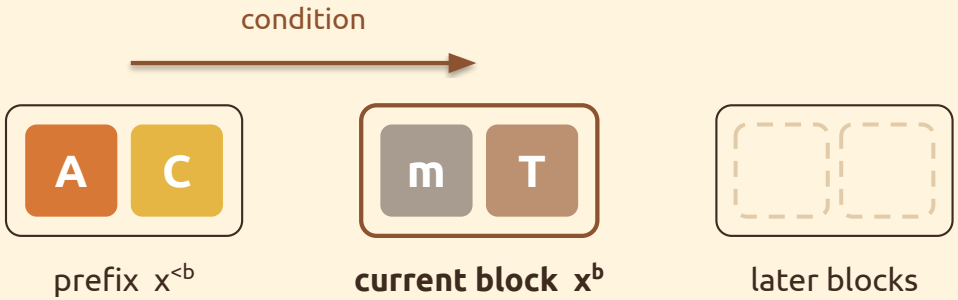
$$p_\theta(x) = \prod_b p_\theta(x^b \mid x^{<b})$$



block size L′ = 2: generate AC, then GT given AC, then AC given ACGT

Each completed block becomes context for the next

$$p_\theta(x) = \prod_b p_\theta(x^b \mid x^{<b})$$



block size L′ = 2: generate AC, then GT given AC, then AC given ACGT

Block Diffusion on ACGTAC

Split this toy DNA sequence into blocks AC, GT and AC. Suppose the model assigns the three required block probabilities 0.4, 0.75 and 0.5, each given its prefix.

Block Diffusion on ACGTAC

Split this toy DNA sequence into blocks AC, GT and AC. Suppose the model assigns the three required block probabilities 0.4, 0.75 and 0.5, each given its prefix.

$$p_\theta(\mathrm{ACGTAC}) = p(\mathrm{AC})p(\mathrm{GT} \mid \mathrm{AC})p(\mathrm{AC} \mid \mathrm{ACGT}) = 0.15$$

Block Diffusion on ACGTAC

Split this toy DNA sequence into blocks AC, GT and AC. Suppose the model assigns the three required block probabilities 0.4, 0.75 and 0.5, each given its prefix.

$$p_\theta(\mathrm{ACGTAC}) = p(\mathrm{AC})p(\mathrm{GT} \mid \mathrm{AC})p(\mathrm{AC} \mid \mathrm{ACGT}) = 0.15$$

The sequence probability is 0.4 times 0.75 times 0.5. Each completed block becomes context.

Block Diffusion on ACGTAC

Split this toy DNA sequence into blocks AC, GT and AC. Suppose the model assigns the three required block probabilities 0.4, 0.75 and 0.5, each given its prefix.

$$p_\theta(\mathrm{ACGTAC}) = p(\mathrm{AC})p(\mathrm{GT} \mid \mathrm{AC})p(\mathrm{AC} \mid \mathrm{ACGT}) = 0.15$$

The sequence probability is 0.4 times 0.75 times 0.5. Each completed block becomes context.

$$-\log(0.15) = -\log(0.4) - \log(0.75) - \log(0.5) \approx 1.897$$

Block Diffusion on ACGTAC

Split this toy DNA sequence into blocks AC, GT and AC. Suppose the model assigns the three required block probabilities 0.4, 0.75 and 0.5, each given its prefix.

$$p_\theta(\mathrm{ACGTAC}) = p(\mathrm{AC})p(\mathrm{GT} \mid \mathrm{AC})p(\mathrm{AC} \mid \mathrm{ACGT}) = 0.15$$

The sequence probability is 0.4 times 0.75 times 0.5. Each completed block becomes context.

$$-\log(0.15) = -\log(0.4) - \log(0.75) - \log(0.5) \approx 1.897$$

The sequence negative log-likelihood is the sum of the three block contributions, in nats.

Block Diffusion on ACGTAC

Split this toy DNA sequence into blocks AC, GT and AC. Suppose the model assigns the three required block probabilities 0.4, 0.75 and 0.5, each given its prefix.

$$p_\theta(\mathrm{ACGTAC}) = p(\mathrm{AC})p(\mathrm{GT} \mid \mathrm{AC})p(\mathrm{AC} \mid \mathrm{ACGT}) = 0.15$$

The sequence probability is 0.4 times 0.75 times 0.5. Each completed block becomes context.

$$-\log(0.15) = -\log(0.4) - \log(0.75) - \log(0.5) \approx 1.897$$

The sequence negative log-likelihood is the sum of the three block contributions, in nats.

Diffusion fills a block while autoregression carries information from earlier blocks.

With blocks, we keep a KV cache and variable length, and we fill each block by diffusion

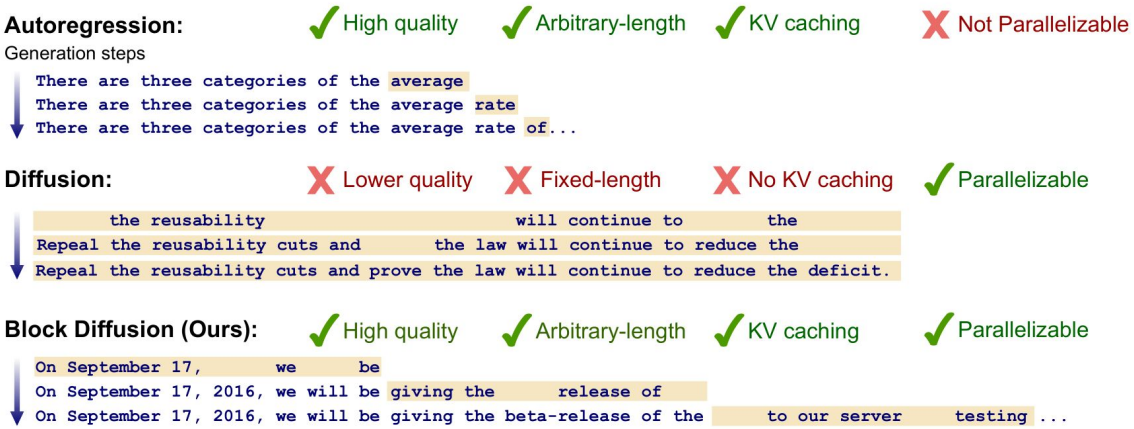


Figure 1: Block diffusion sequentially generates blocks of tokens by performing diffusion within each block and conditioning on previous blocks. By combining strength from autoregressive and diffusion models, block diffusion overcomes the limitations of both approaches by supporting variable-length, higher-quality generation and improving inference efficiency with KV caching and parallel sampling.

In training, we use two forward passes, one for the clean prefix and one for the noisy block.

Arriola,..,Kuleshov. *ICLR* (2025)

Here is the test perplexity on LM1B in Table 3 of the paper

Table 3: Test perplexities (PPL; ↓) of models trained for 65B tokens on LM1B. Best diffusion value is bolded.

	PPL (↓)
Autoregressive	
Transformer-X Base (Dai et al., 2019)	23.5
Transformer (Sahoo et al., 2024a)	22.83
Diffusion	
D3PM (absorb) (Austin et al., 2021)	$\leq$ 82.34
SEDD (Lou et al., 2024)	$\leq$ 32.68
MDLM (Sahoo et al., 2024a)	$\leq$ 31.78
Block diffusion (Ours)	
BD3-LMs $L' = 16$	$\leq$ 30.60
$L' = 8$	$\leq$ 29.83
$L' = 4$	$\leq$ **28.23**

Arriola,..,Kuleshov. *ICLR* (2025)

A retrained transformer reaches 22.83 on LM1B

Table 3: Test perplexities (PPL; ↓) of models trained for 65B tokens on LM1B. Best diffusion value is bolded.

	PPL (↓)
Autoregressive	
Transformer-X Base (Dai et al., 2019)	23.5
Transformer (Sahoo et al., 2024a)	22.83
Diffusion	
D3PM (absorb) (Austin et al., 2021)	$\leq$ 82.34
SEDD (Lou et al., 2024)	$\leq$ 32.68
MDLM (Sahoo et al., 2024a)	$\leq$ 31.78
Block diffusion (Ours)	
BD3-LMs $L' = 16$	$\leq$ 30.60
$L' = 8$	$\leq$ 29.83
$L' = 4$	$\leq$ **28.23**

Arriola,..,Kuleshov. *ICLR* (2025)

MDLM has a perplexity bound of 31.78 with the same budget

Table 3: Test perplexities (PPL; ↓) of models trained for 65B tokens on LM1B. Best diffusion value is bolded.

	PPL (↓)
Autoregressive	
Transformer-X Base (Dai et al., 2019)	23.5
Transformer (Sahoo et al., 2024a)	22.83
Diffusion	
D3PM (absorb) (Austin et al., 2021)	$\leq$ 82.34
SEDD (Lou et al., 2024)	$\leq$ 32.68
MDLM (Sahoo et al., 2024a)	$\leq$ 31.78
Block diffusion (Ours)	
BD3-LMs $L' = 16$	$\leq$ 30.60
$L' = 8$	$\leq$ 29.83
$L' = 4$	$\leq$ **28.23**

Arriola,..,Kuleshov. *ICLR* (2025)

Blocks of four tokens reach 28.23, between the two

Table 3: Test perplexities (PPL; ↓) of models trained for 65B tokens on LM1B. Best diffusion value is bolded.

	PPL (↓)
Autoregressive	
Transformer-X Base (Dai et al., 2019)	23.5
Transformer (Sahoo et al., 2024a)	22.83
Diffusion	
D3PM (absorb) (Austin et al., 2021)	$\leq$ 82.34
SEDD (Lou et al., 2024)	$\leq$ 32.68
MDLM (Sahoo et al., 2024a)	$\leq$ 31.78
Block diffusion (Ours)	
BD3-LMs $L' = 16$	$\leq$ 30.60
$L' = 8$	$\leq$ 29.83
$L' = 4$	$\leq$ **28.23**

Arriola,..,Kuleshov. *ICLR* (2025)

With smaller blocks, the bound moves toward autoregression

Table 3: Test perplexities (PPL; ↓) of models trained for 65B tokens on LM1B. Best diffusion value is bolded.

	PPL (↓)
Autoregressive	
Transformer-X Base (Dai et al., 2019)	23.5
Transformer (Sahoo et al., 2024a)	22.83
Diffusion	
D3PM (absorb) (Austin et al., 2021)	$\leq$ 82.34
SEDD (Lou et al., 2024)	$\leq$ 32.68
MDLM (Sahoo et al., 2024a)	$\leq$ 31.78
Block diffusion (Ours)	
BD3-LMs $L' = 16$	$\leq$ 30.60
$L' = 8$	$\leq$ 29.83
$L' = 4$	$\leq$ **28.23**

Arriola,..,Kuleshov. *ICLR* (2025)